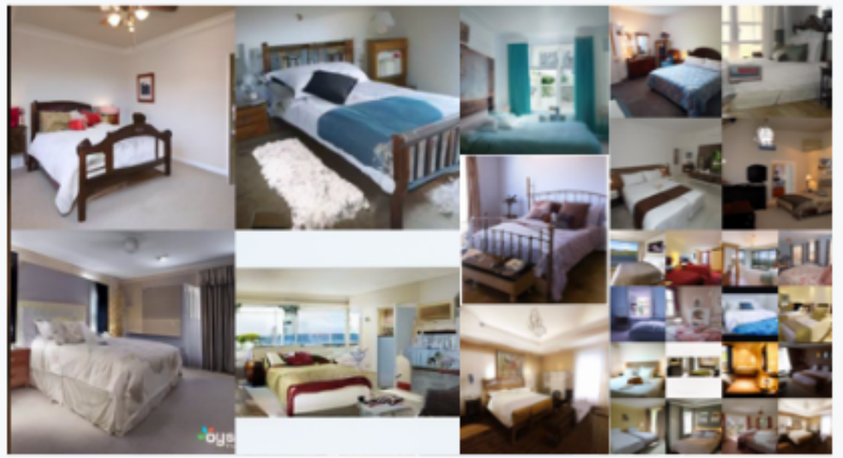
Can we create a new example?

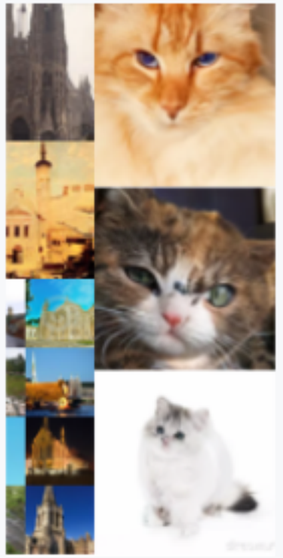
Faces



Rooms



Cats



Published unconditional samples · Ho, Jain & Abbeel (2020)

Takeaway A generator learns from examples so that it can produce new samples.

One description, several valid images

"A basket full of cherries"



Sample 1



Sample 2



Sample 3



Sample 4

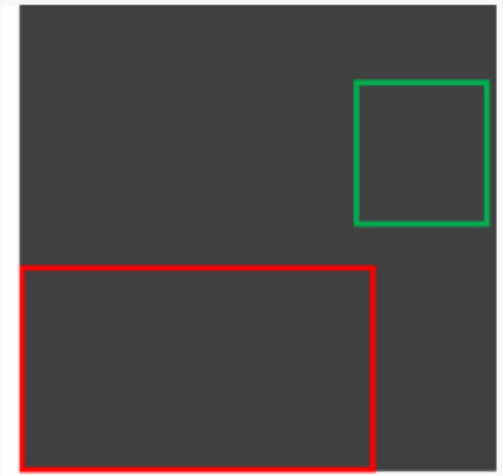
Published LDM outputs · the source does not specify matched random seeds

Takeaway The prompt describes a distribution of possible images, so different samples can all fit.

Where should the objects go?

"Space view of a planet and its sun"

Supplied layout



Published samples



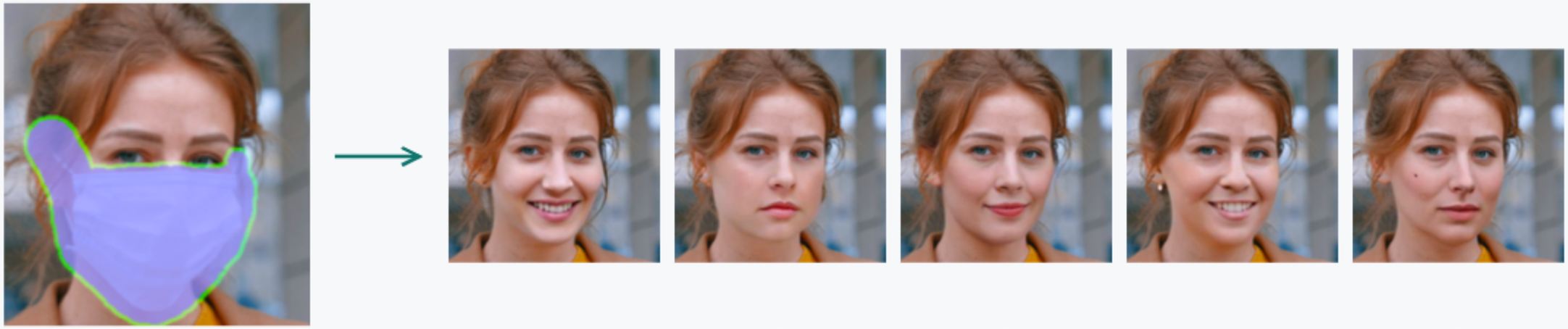
GLIGEN · red box: planet · green box: sun

Takeaway A layout constrains object positions while leaving room for different appearances.

Fill a missing region

Observed pixels

Five possible fillings of the same missing region



RePaint · blue marks the missing region · outputs are samples, not successive steps

Takeaway The visible pixels constrain the answer, but several completions can still make sense.

Turn rough strokes into a plausible image

Input strokes



Published output 1



Published output 2



SDEdit · the supplied image sets the starting point before corruption and denoising

Takeaway More initial noise gives the model more freedom and can weaken the influence of the strokes.

What detail is missing at low resolution?



SR3 source comparison · the lower row enlarges the eye region

Takeaway Generated detail may look convincing without recovering the detail in the original scene.

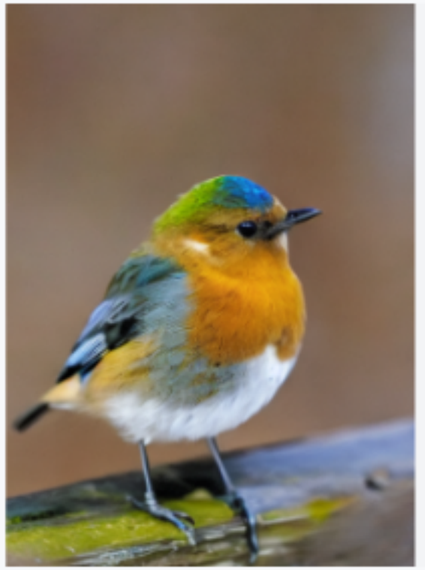
Draw the structure, generate the appearance

Canny edge condition



ControlNet →

Generated image



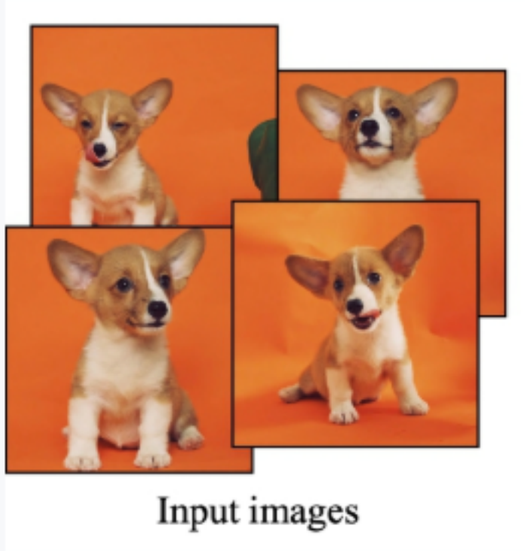
Published input–output pair · the edge map supplies spatial structure

Takeaway Spatial features can serve as conditions that guide where image structure appears.

Put a particular subject in a new context

ADAPTATION

GENERATION
New prompts



Adapted model
subject-specific weights



At the Acropolis



Getting a haircut

DreamBooth · reference images enter during adaptation

Takeaway DreamBooth adapts a model to a subject; prompts then guide images made by that adapted model.

Could this help answer a lab question?

Published DiffusionSat samples

"An electric substation in Finland"

August

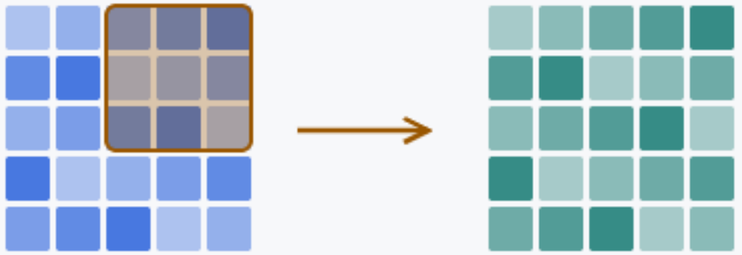


January



Same caption; month condition changed

POSSIBLE LAB TASK



Partial observation

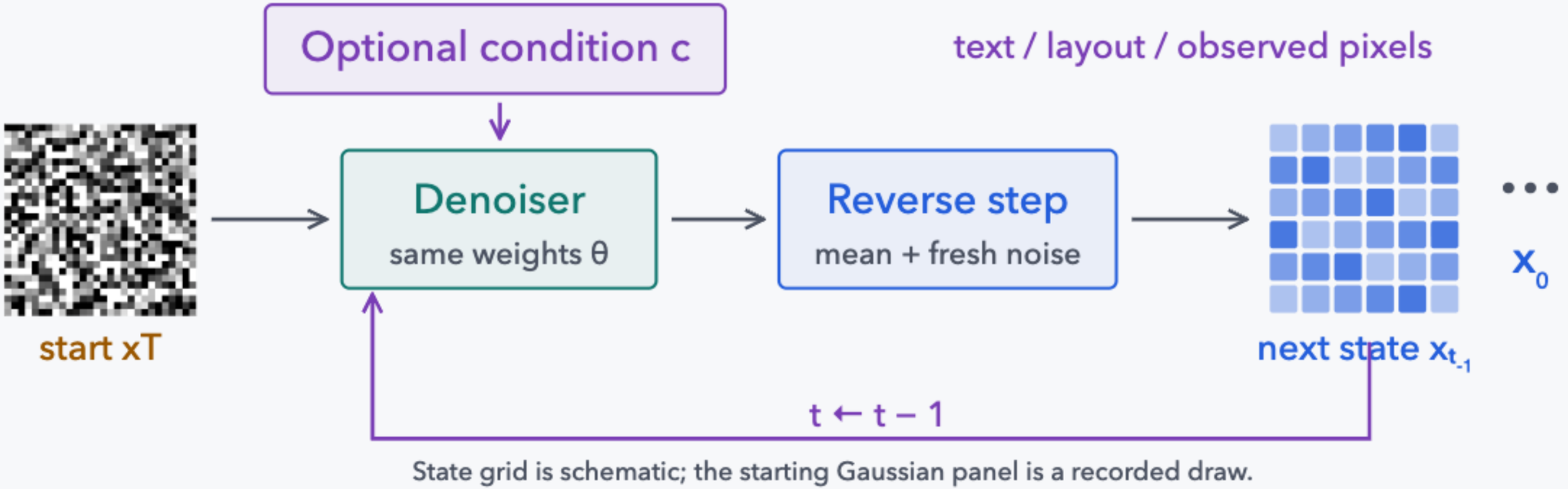
Candidate filling

Compare with withheld measurements

Proposed task; no local result claimed

Takeaway Scientific usefulness needs validation against observations, beyond a convincing image.

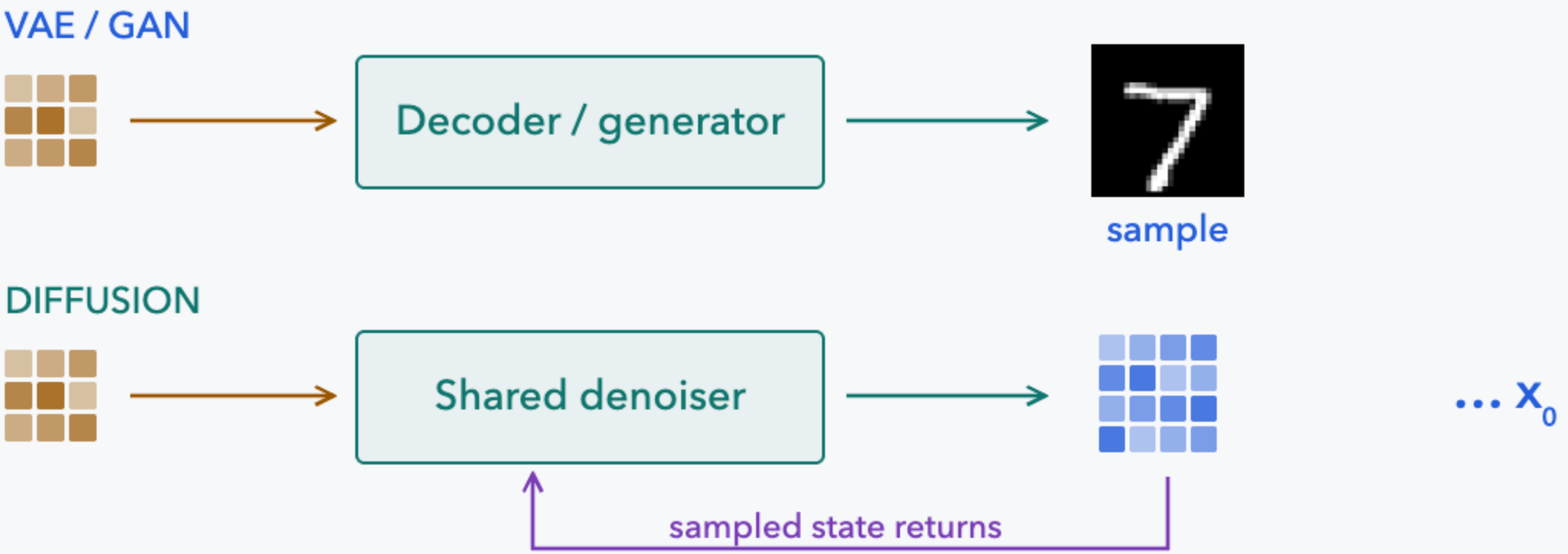
What must a generator learn?



State grid is schematic; the starting Gaussian panel is a recorded draw.

Takeaway We need a training example, a prediction target and a rule that turns predictions into samples.

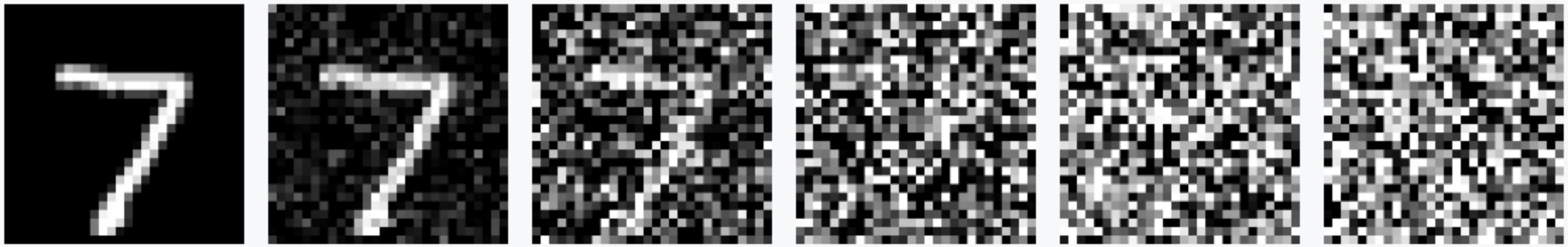
From one-step generators to a denoising chain



Takeaway Diffusion generates through a chain of denoising steps that share one learned model.

A digit gradually loses information

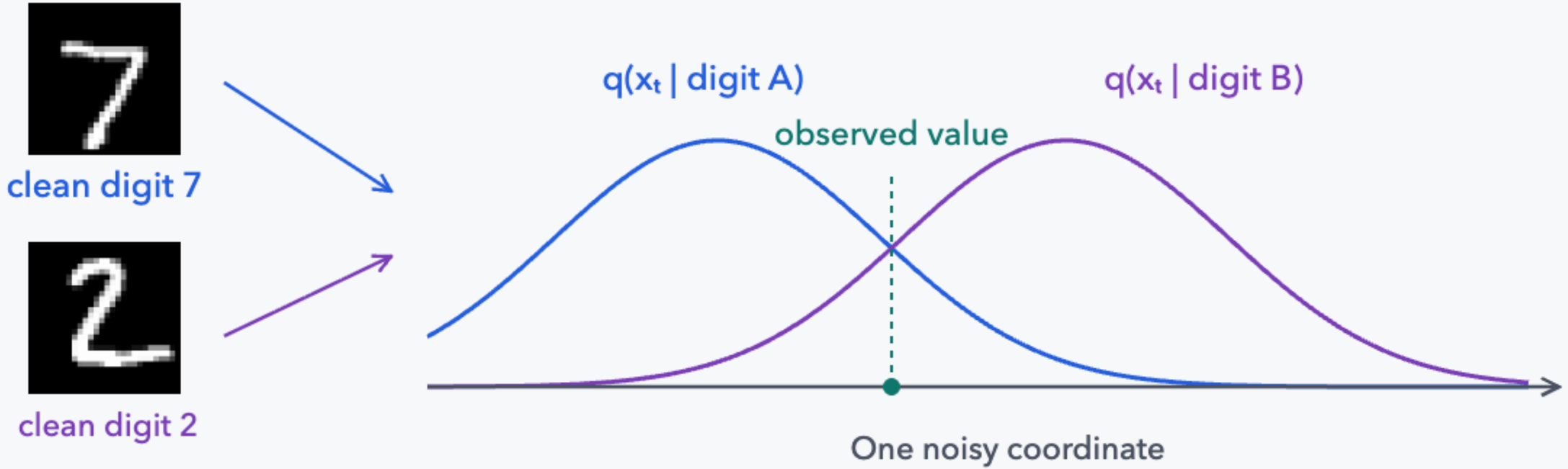
Same digit · one recorded recursive trajectory



MNIST test image 0 · seed 667 · fixed display range [−1,1]

Takeaway Repeated corruption gradually makes the original digit harder to identify.

Can we recover the original with certainty?



Illustrative overlapping densities; the two full image arrays are not asserted to be equal.

Takeaway Denoising uses learned patterns to handle the uncertainty left by corruption.

What happens to a whole distribution?

Time

Seed 667

Process Forward marginals

Reset Step Play Pause




```
t = 0; seed 667
8 centres on radius 2
Within-cluster τ = .18
Marginal variance 0.03240
εoracle = −√(1−ᾱ) score
Fixed ε: direct marginal
coupling.
Axes remain fixed across time.
```

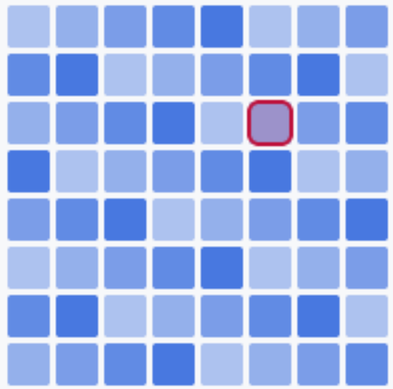
Analytical score oracle for a known toy distribution; no neural network was trained.

Takeaway Noise changes a distribution; this toy example lets us calculate its score analytically.

What is a clean image tensor?



MNIST digit 7



28 × 28 positions

One coordinate

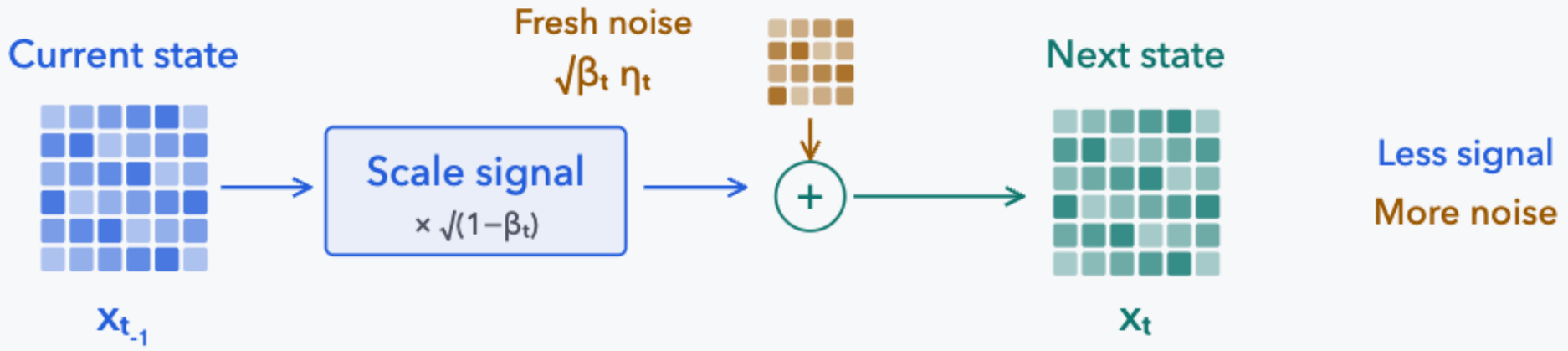
uint8 / 127.5 − 1



Batch shape [B,1,28,28] · one channel per image

Takeaway Our clean pixels lie in [−1, 1], but adding Gaussian noise can take values outside that range.

One small corruption step

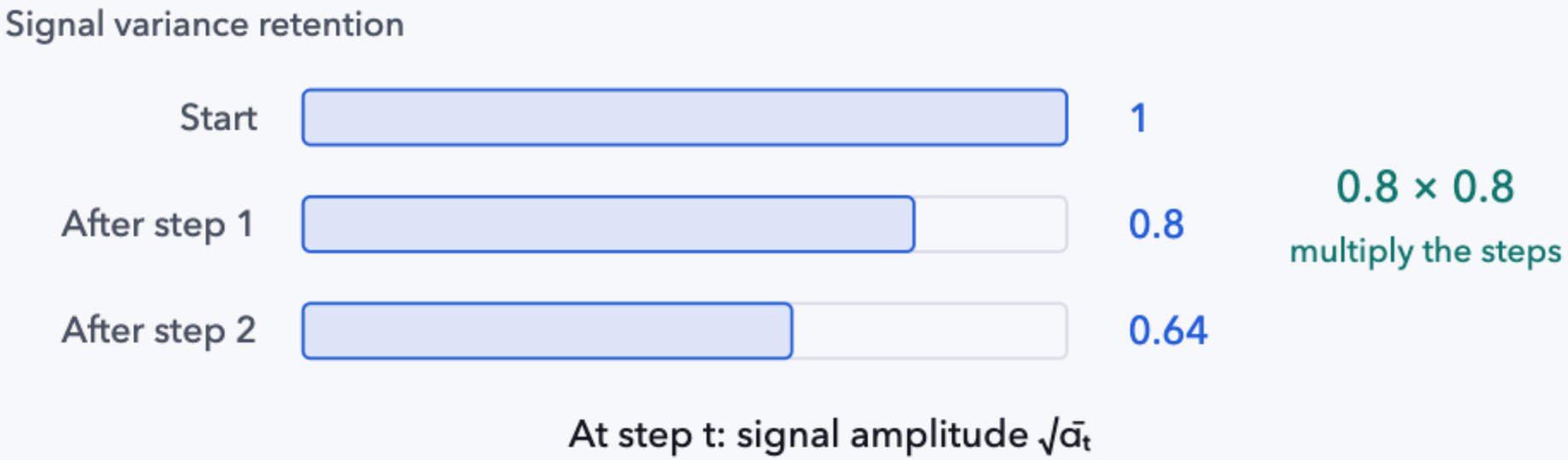


Tensor cells are schematic; the equation defines the actual operation.

$$x_t = \sqrt{1-\beta_t}\,x_{t-1} + \sqrt{\beta_t}\,\eta_t$$

Takeaway Beta specifies the added variance, so the noise amplitude is the square root of beta.

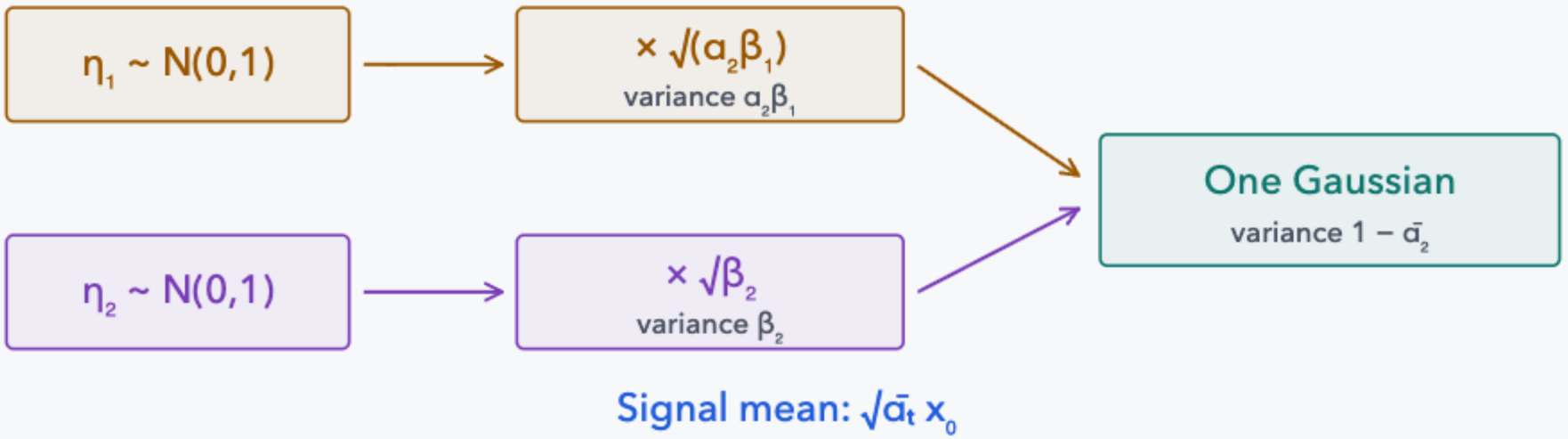
Alpha and alpha-bar mean different things



$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s, \quad \bar{\alpha}_0 = 1$$

Takeaway Alpha-bar is the product of the stepwise alphas and tracks cumulative signal variance retention.

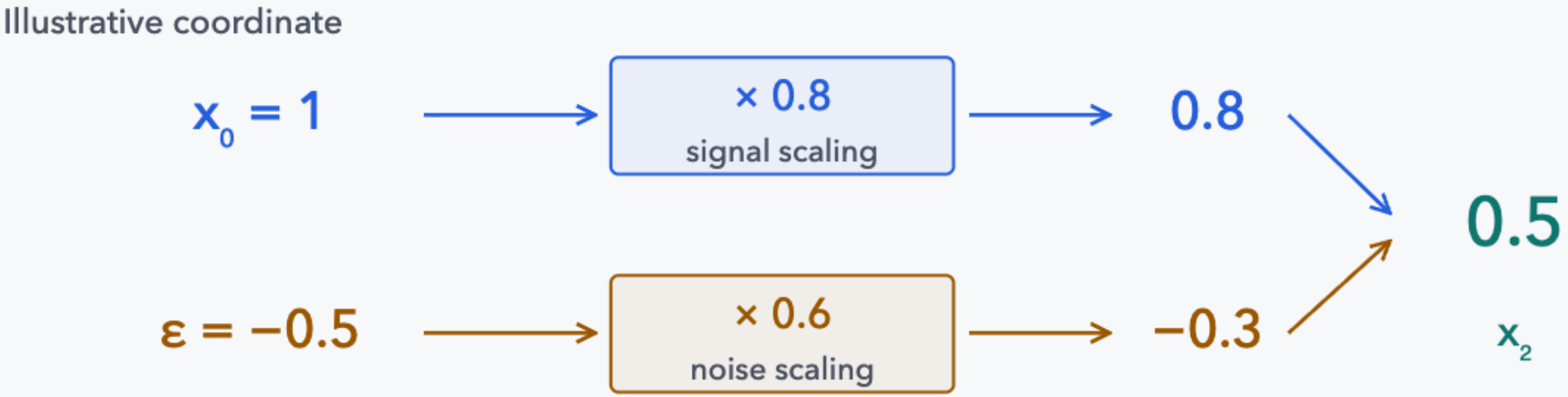
Jump directly to a training noise level



$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Takeaway We can sample a noisy training input at any chosen time without simulating all earlier steps.

Work through one noisy coordinate



Takeaway The signal and noise contributions can cancel; a noisy value alone does not identify either contribution.

Try changing the noise level

Timestep

Seed 667

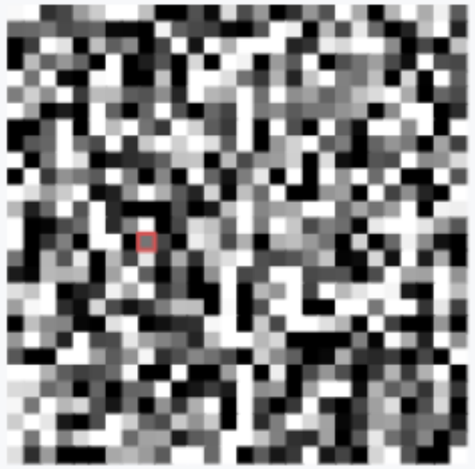
Coordinate 400

Process: Direct marginal, shared ε

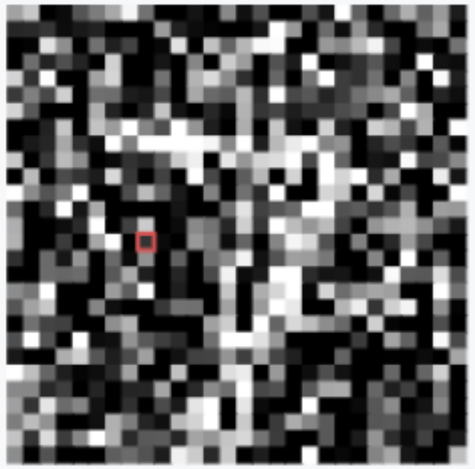
Reset | Step | Play | Pause | Resample noise | Schedule / SNR



Clean x0



Aggregate ε



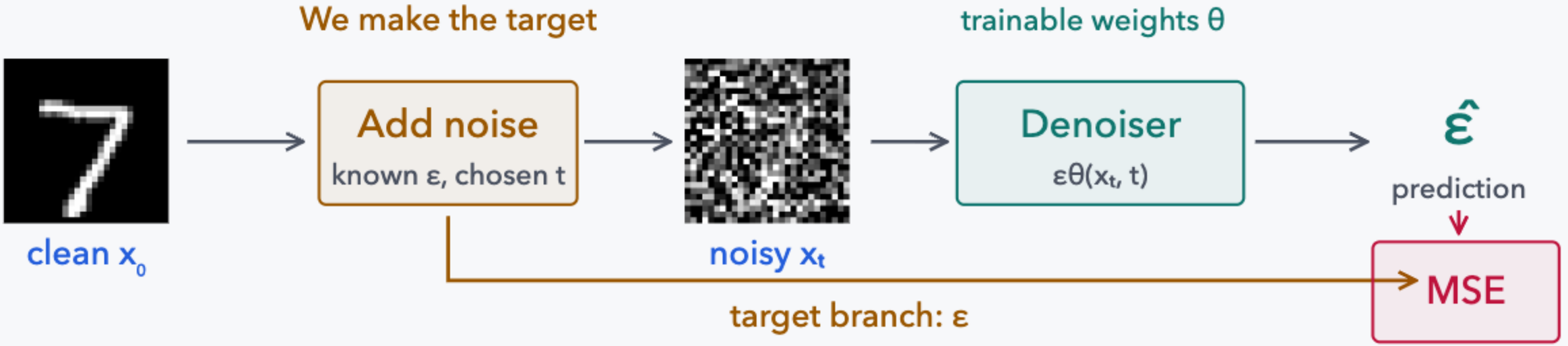
x350

```
t = 350
β = 0.007052
α = 0.992948
ᾱ = 0.285200
Signal 0.5340  Noise 0.8455
SNR = 0.3990
Pixel 400: x0=-1.000
xt=-0.6050
Shared ε marginal coupling
```

Computed Gaussian corruption · fixed pixel scale [-1,1] for display only

Takeaway Direct sampling gives the correct distribution at each time; a recursive path also specifies how times relate.

Which target should the network predict?

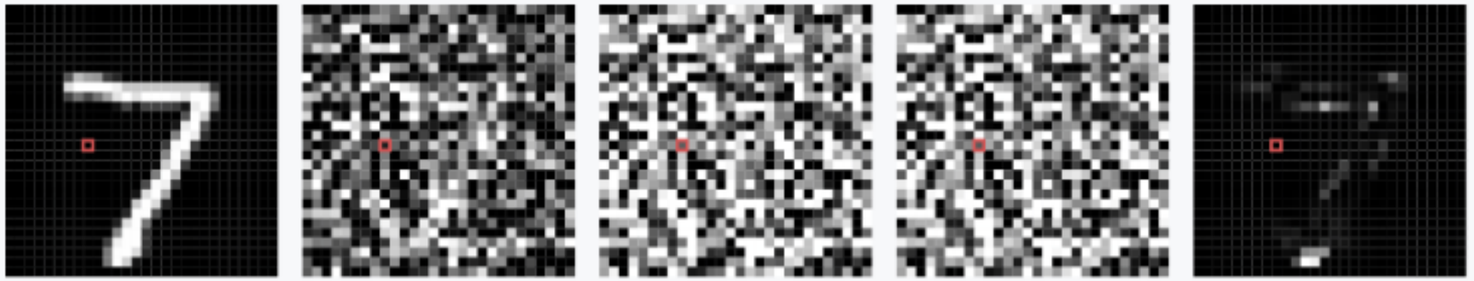


Takeaway The denoiser takes a noisy image and its timestep, then predicts a real-valued noise tensor.

Assemble one training example

Digit index: 0 | Recorded t: 350 | Recorded seed: 667 | Coordinate: 400

Reset | Step | Play | Pause | Make batch of 3



x0 | xt | target ε | predicted ε | error² [0,1]

`[B,1,28,28]` `Target ε: −0.2740` `Prediction: −0.3019` `Squared error: 0.000783`

`Image MSE: 0.021368` `B=1, t=350` `model(xt,t) → ε̂ → mean(error²)`

Actual predictions replayed from the new trained checkpoint; controls select recorded tuples.

Takeaway The model sees only the noisy image and time; the sampled noise stays on the target side.

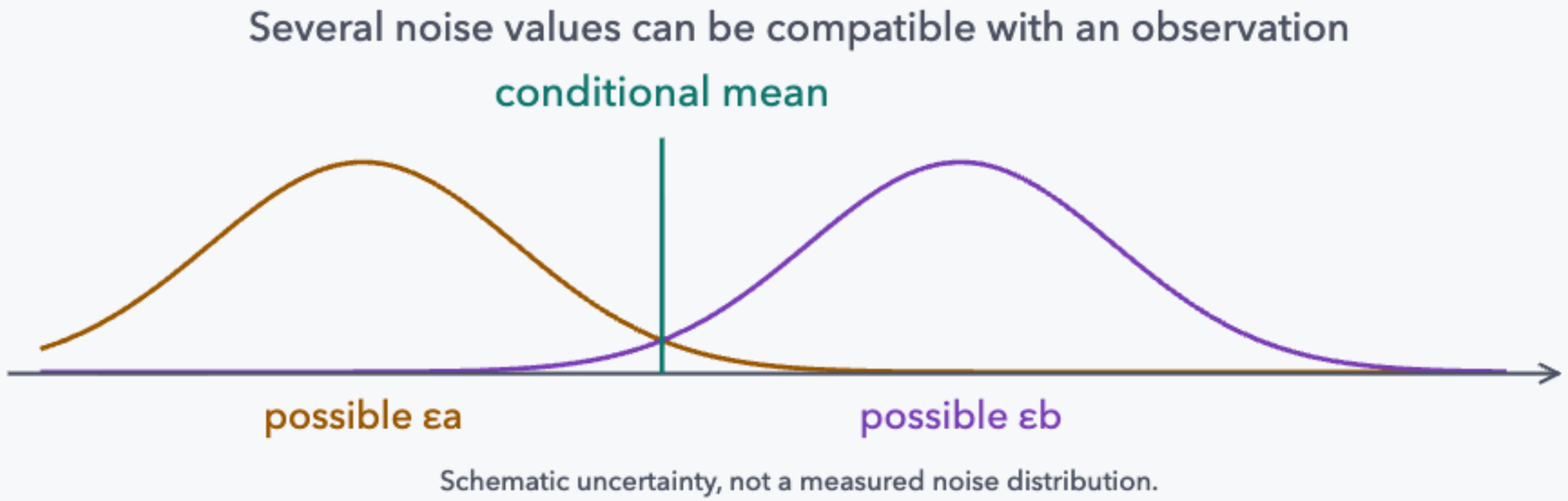
Calculate the loss for four values



$$0.15 \div 4 = 0.0375$$

Takeaway Averaging over tensor entries turns the prediction errors into one scalar training loss.

Can the network know the exact sampled noise?

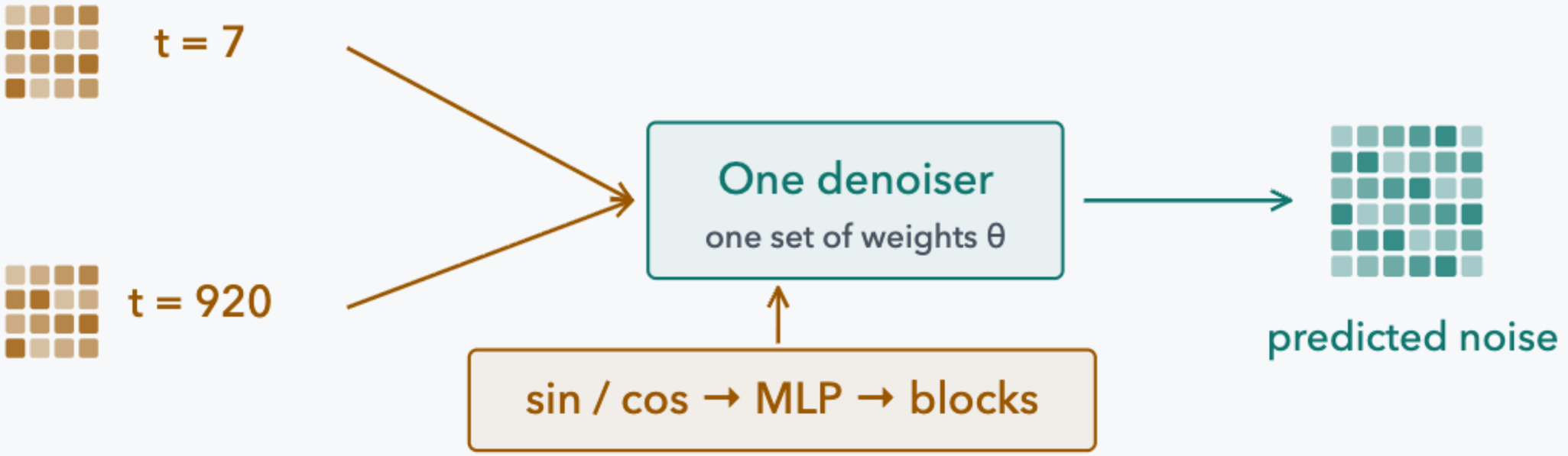


Schematic uncertainty, not a measured noise distribution.

$$\epsilon_\theta^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t, t]$$

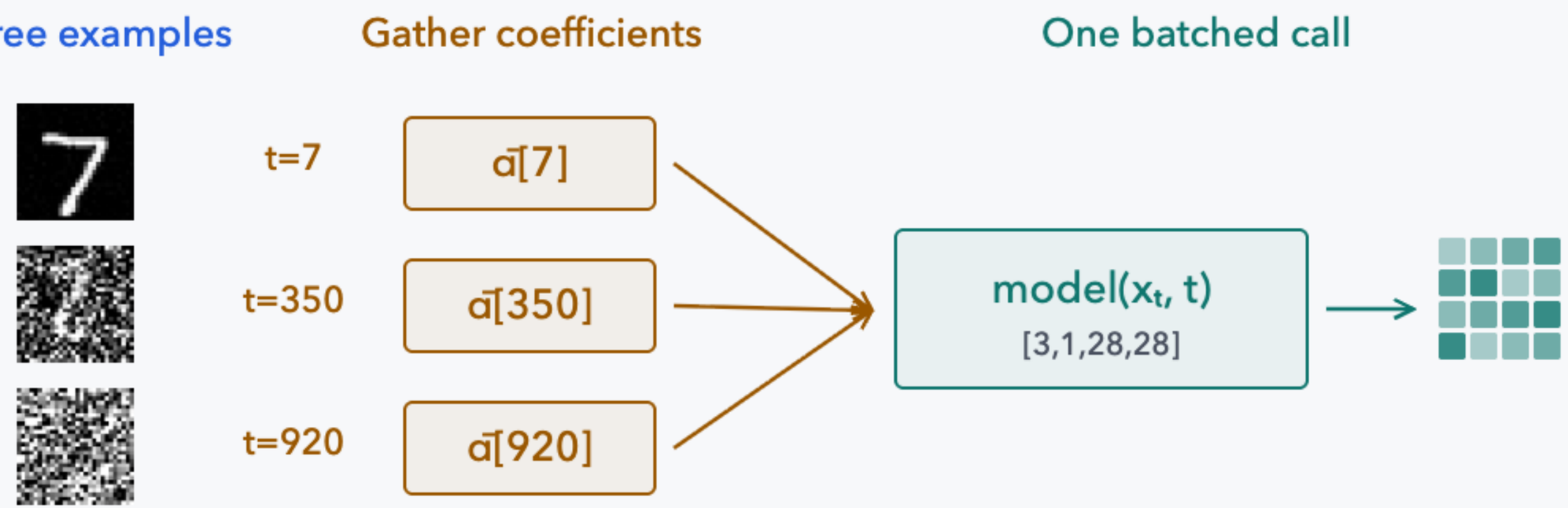
Takeaway Under squared error, the best population prediction is the conditional mean of the possible noise values.

One denoiser, many noise levels



Takeaway A time embedding lets one set of weights handle many corruption levels.

A batch can contain different timesteps

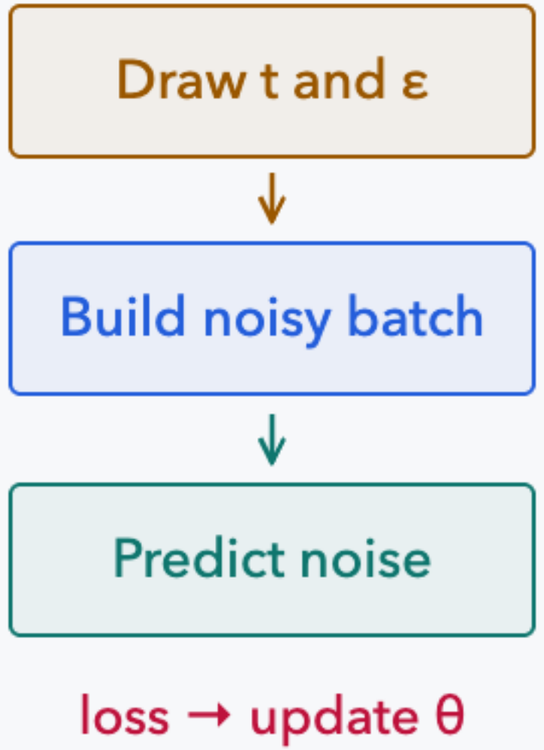


Gather [B] → reshape [B,1,1,1] → broadcast only across that example

Takeaway The batch dimension binds each image to its own timestep and noise draw.

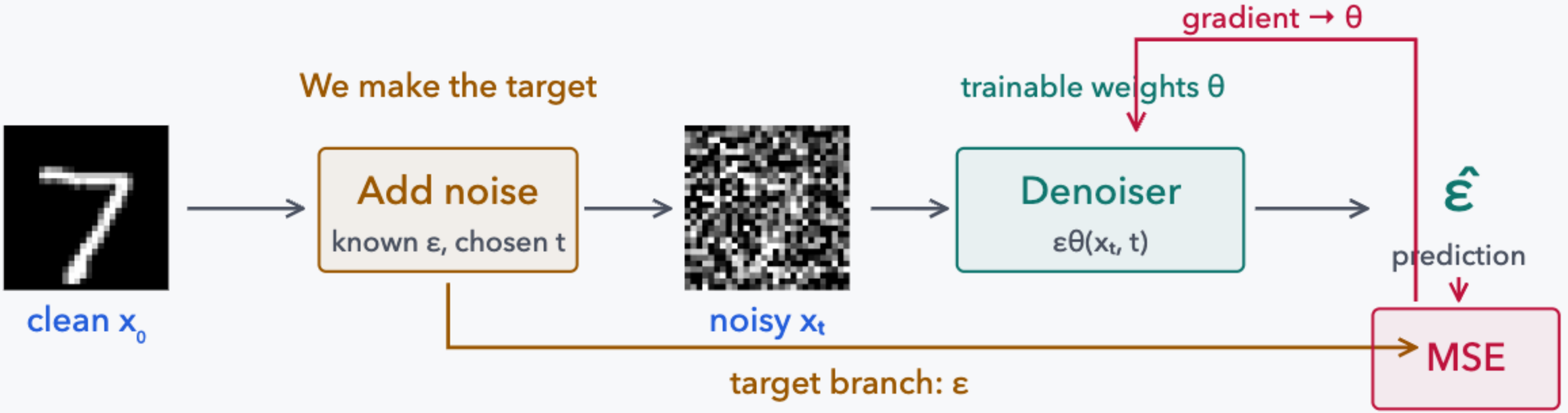
Translate the picture into PyTorch

```
t = torch.randint(1, T+1, (B,), device=x0.device)
eps = torch.randn_like(x0)
ab = alpha_bar[t].reshape(B, 1, 1, 1)
xt = ab.sqrt()*x0 + (1-ab).sqrt()*eps
eps_hat = model(xt, t)
loss = F.mse_loss(eps_hat, eps)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```



Takeaway One training update needs one denoiser call on a batch of directly constructed noisy inputs.

Where do the gradients go?



Takeaway Training updates the denoiser weights; it does not run the full generation chain.

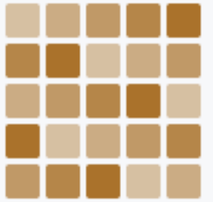
Where do the training labels come from?

Clean examples



The data determine coverage

→ **Draw Gaussian noise**
known numerical target →

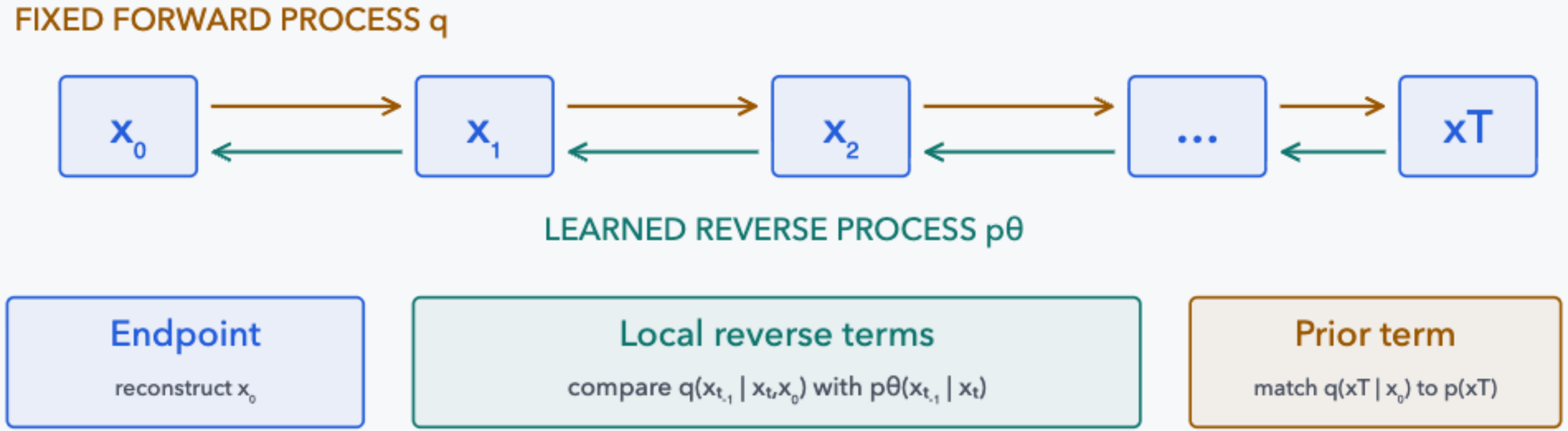


If training contains only sevens…

extra corruption draws do not supply examples of other digits.

Takeaway We can create noise targets automatically, but the clean dataset still determines what the model can learn.

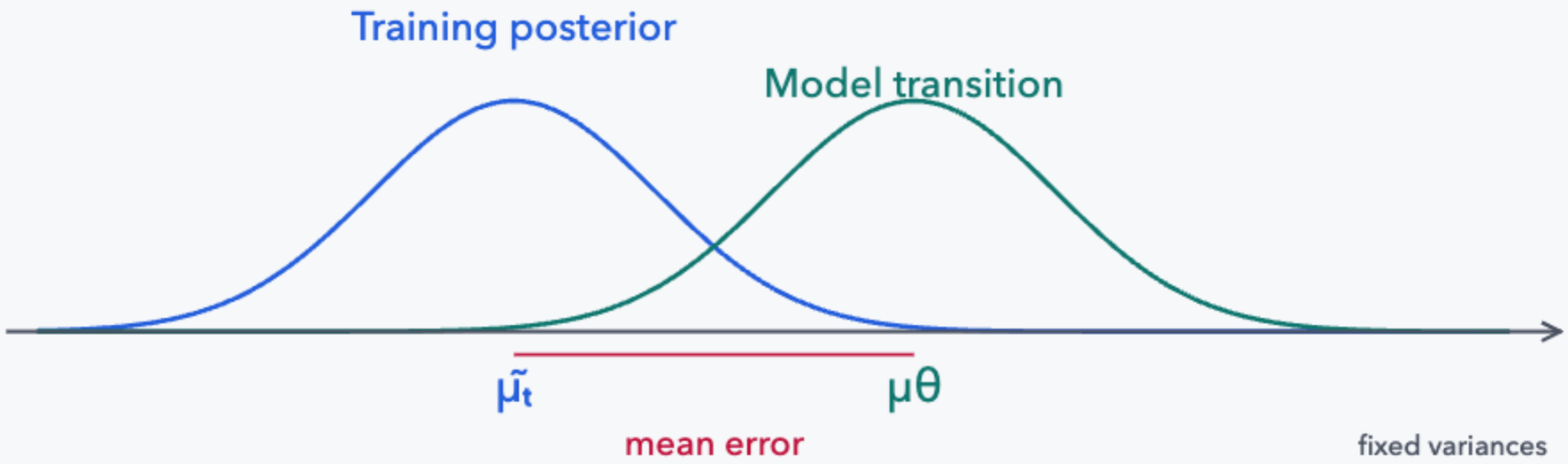
The ELBO from Lecture I appears again



$$p_\theta(x_{0:T}) = p(x_T) \prod_t p_\theta(x_{t-1} \mid x_t)$$

Takeaway The diffusion bound combines a prior term, local reverse-matching terms and an endpoint likelihood.

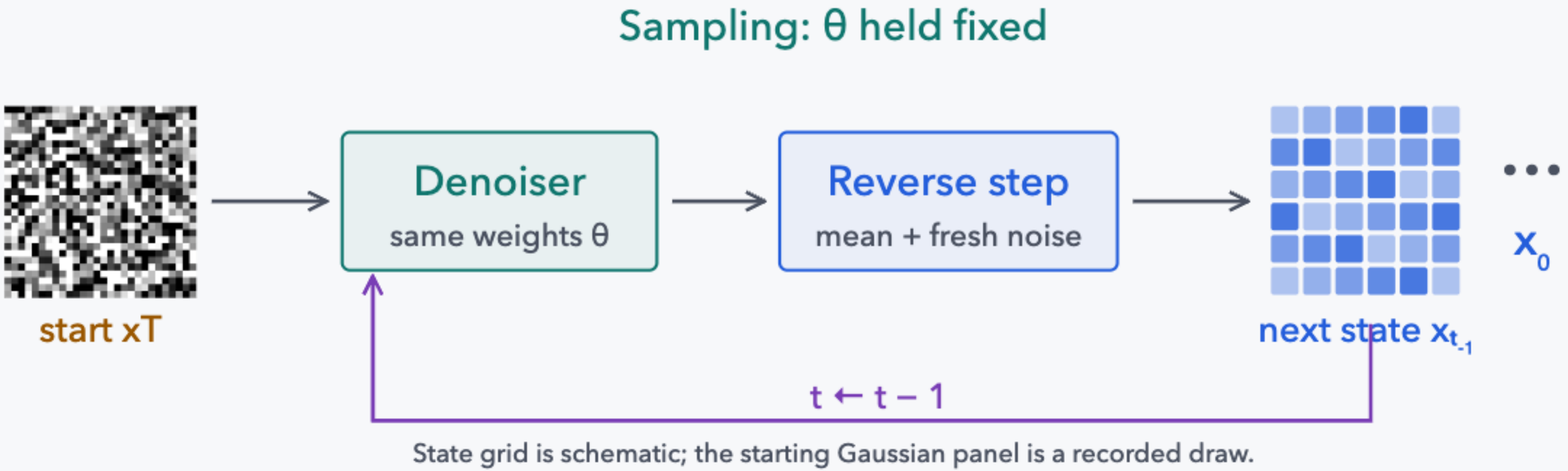
How does the bound lead to noise MSE?



$$\mathcal{L}_{simple} = \mathbb{E}_{x_0,t,\epsilon}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

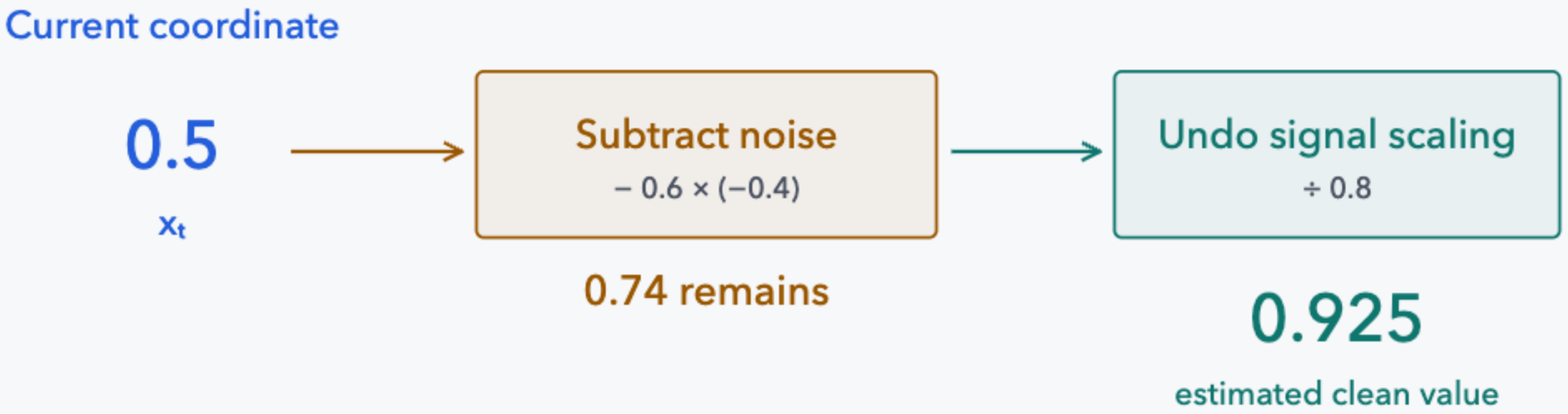
Takeaway Simple noise MSE changes the variational weighting and does not include every term of the negative ELBO.

Sampling starts without a clean image



Takeaway Generation keeps the learned weights fixed and repeatedly samples a less noisy state.

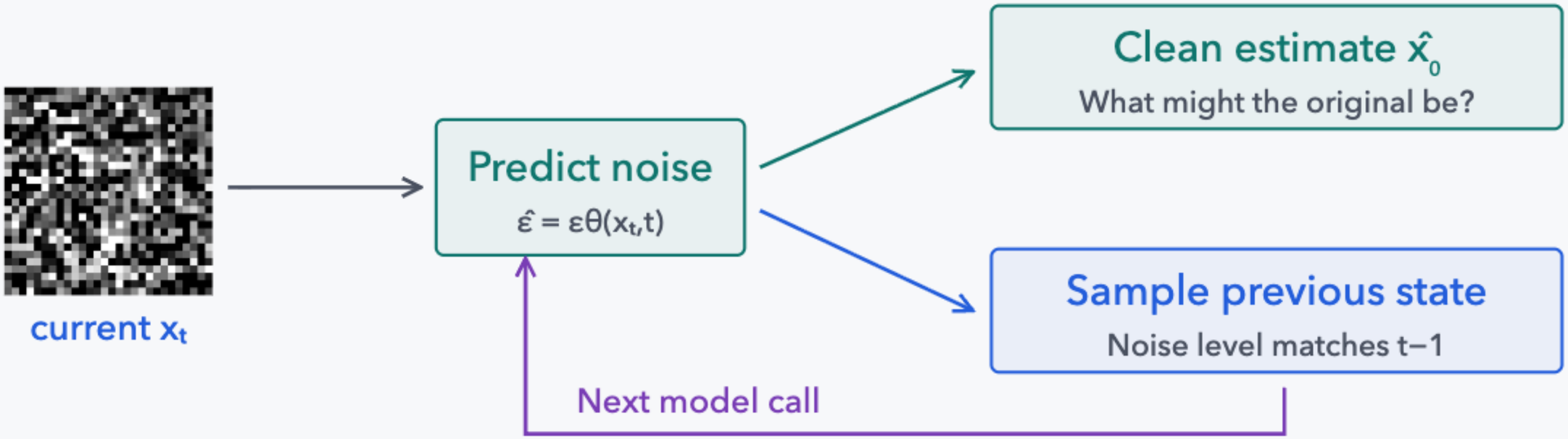
A predicted clean image is useful



$$\hat{x}_0 = \frac{.5 - .6(-.4)}{.8} = .925$$

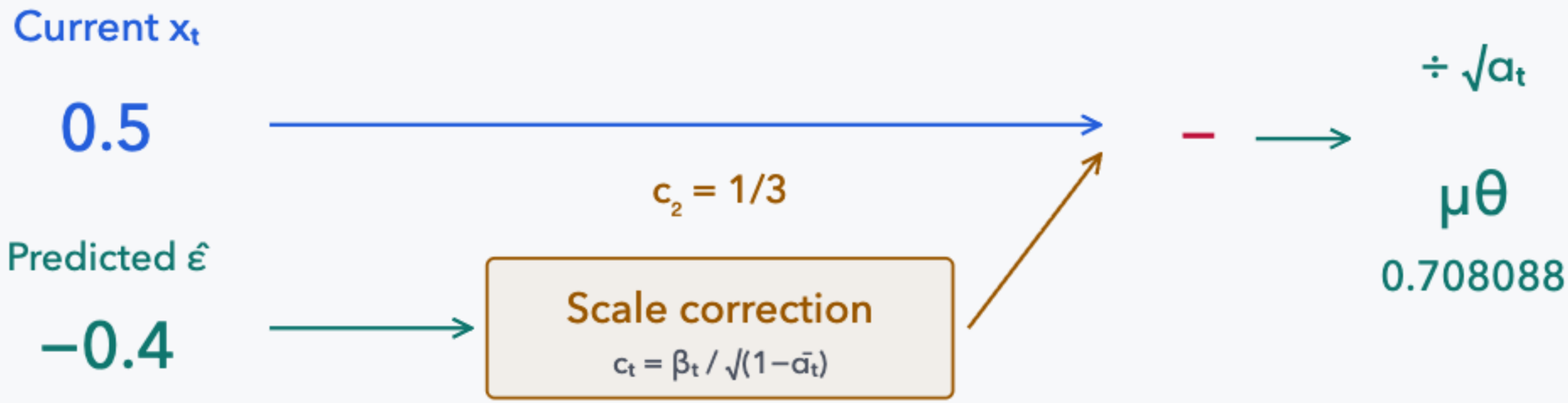
Takeaway The predicted noise gives a clean-image estimate, though small signal coefficients can amplify its error.

Which state goes into the next call?



Takeaway The next denoiser call receives the sampled previous state, with the noise level expected at that time.

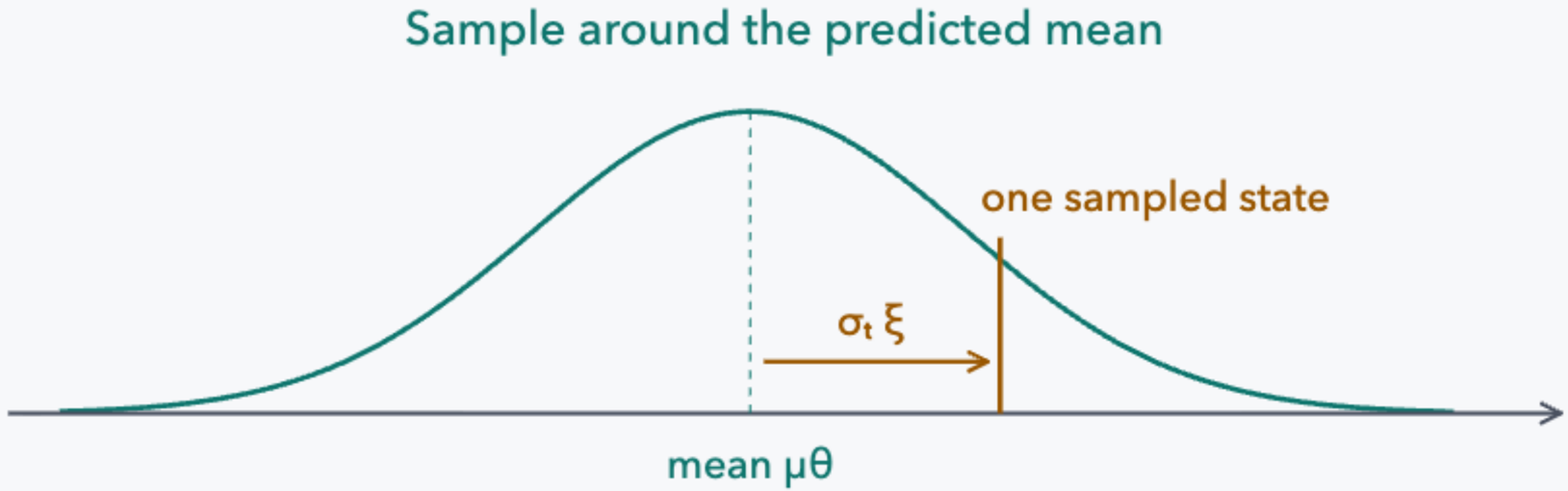
Build one reverse transition



$$\mu_\theta = \frac{x_t - c_t \hat{\epsilon}}{\sqrt{\alpha_t}}$$

Takeaway The reverse mean uses one-step alpha; alpha-bar appears in the correction for predicted noise.

Why add noise during denoising?



Schematic Gaussian transition; the spread comes from the chosen reverse variance.

$$x_{t-1} = \mu_\theta + \mathbf{1}[t > 1]\sqrt{\tilde{\beta}_t}\xi$$

Takeaway This sampler adds noise according to the chosen reverse variance, except at the final step.

Work through one reverse step

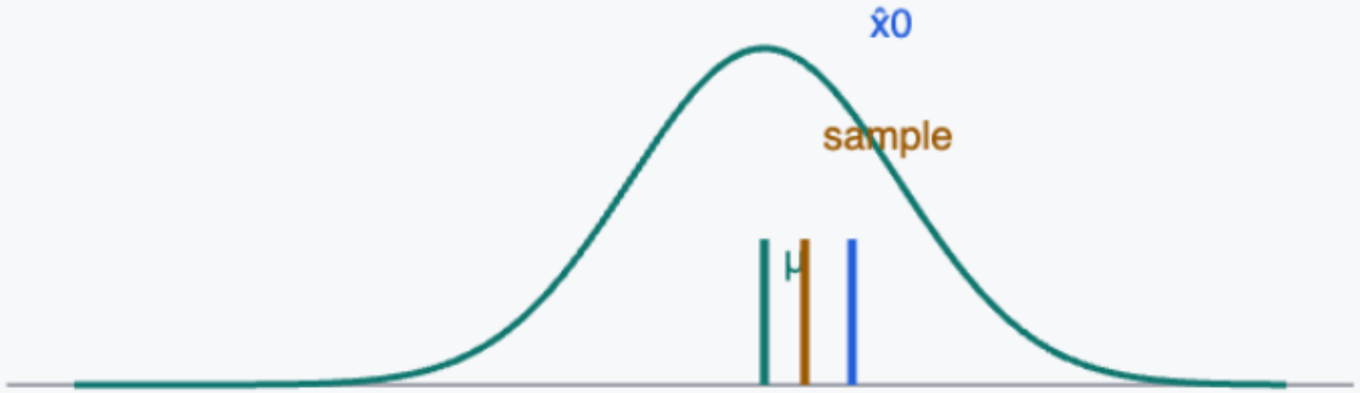
Predicted ε

Fresh z

Transition

t=2 → 1

Reset | Step | Play | Pause



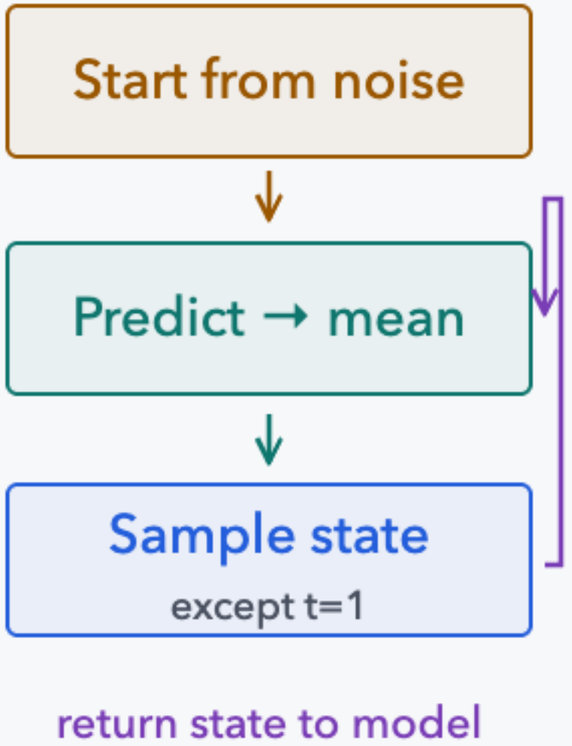

```
ε̂ = −0.4; z = 0.3
x̂0 = 0.925000
μ = 0.708088
variance = 0.111111
std dev = 0.333333
sample = 0.808088
Sample = μ + σz
```

Arithmetic illustration · β1=β2=.2; parameters remain internally consistent.

Takeaway Fresh sampling noise changes the sampled state while leaving the transition mean unchanged.

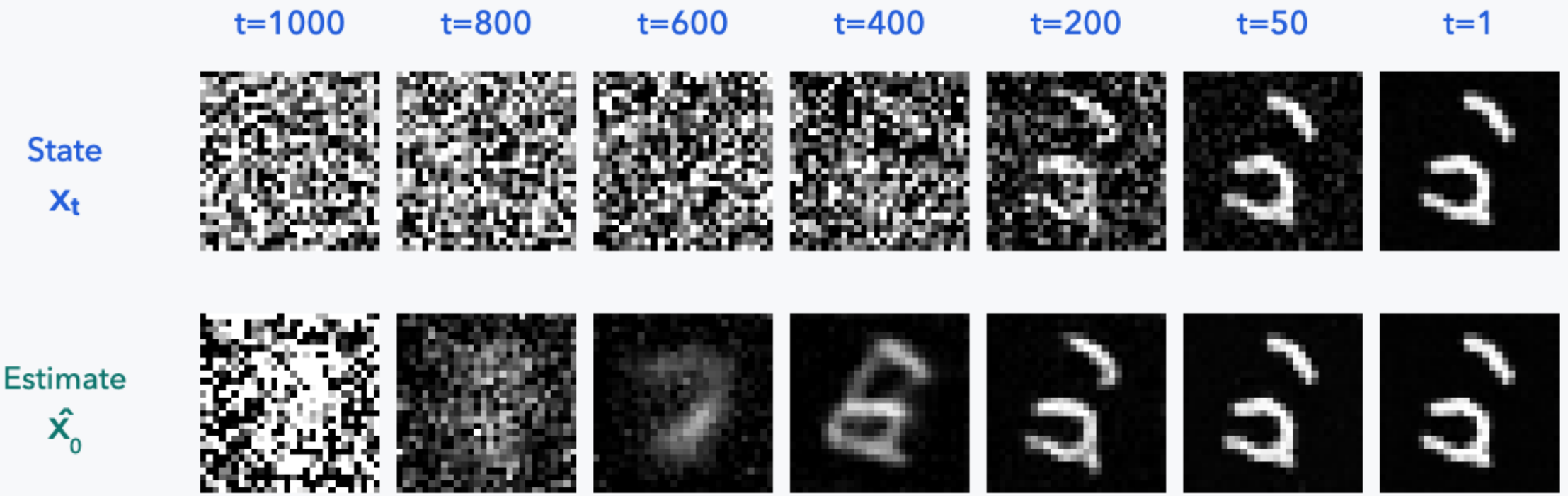
The complete sampling loop

```
x = torch.randn(shape)
for step in range(T, 0, -1):
    t = torch.full((B,), step, dtype=torch.long)
    eps_hat = model(x, t)
    mean = (x - beta[step] * eps_hat
            / (1-alpha_bar[step]).sqrt())
    mean = mean / alpha[step].sqrt()
    if step > 1:
        x = mean + posterior_var[step].sqrt() * randn()
    else:
        x = mean
```



Takeaway The same trained denoiser is reused from T down to 1, with no extra noise in the last step.

What are we watching in a generation video?

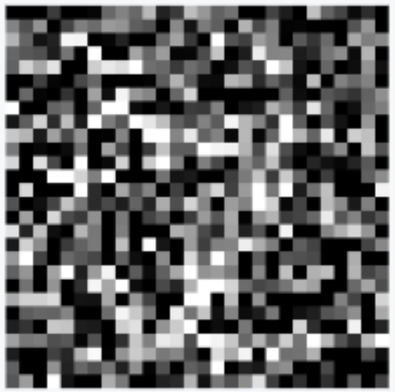


Actual checkpoint replay · seed 667 · display clipping only

Takeaway A noisy chain state and its clean-image estimate come from the same call but serve different purposes.

What should the denoiser output?

The spatial shape stays [B,1,28,28]



noisy image x_t

Dense predictor

one output per position

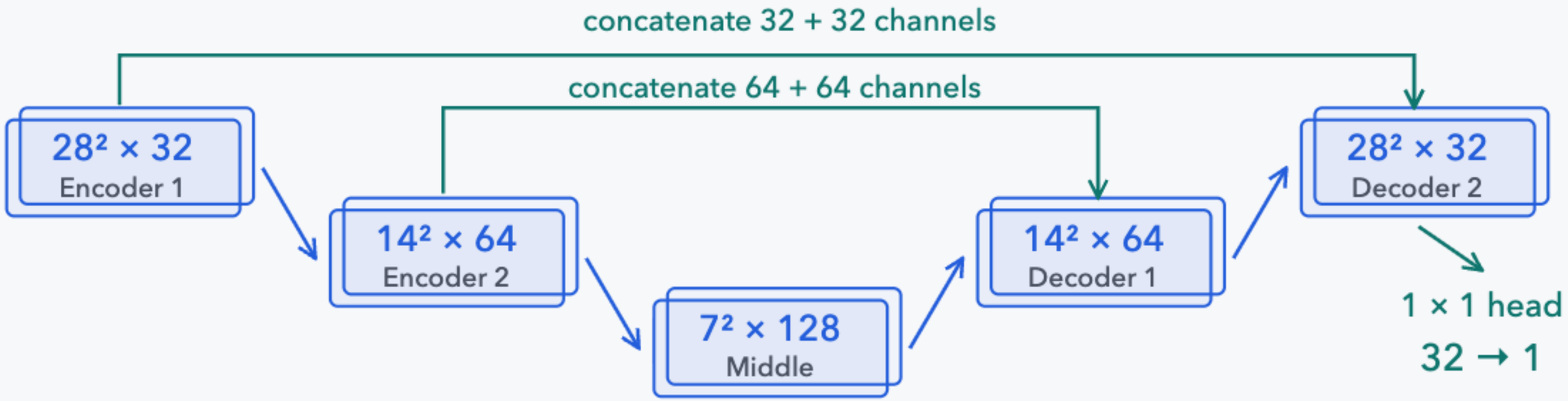


predicted noise $\hat{\epsilon}$

Recorded input and prediction from the same t=350 model call.

Takeaway Noise prediction is a dense regression task; a U-Net is one way to implement it.

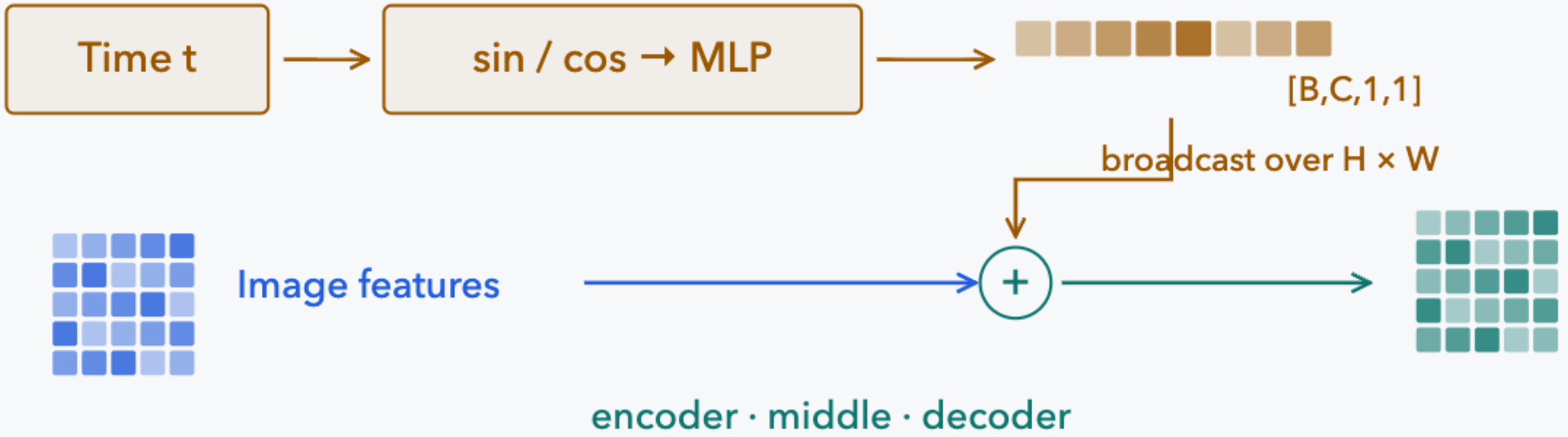
Follow the shapes through a small U-Net



814,145 trainable parameters in this implementation

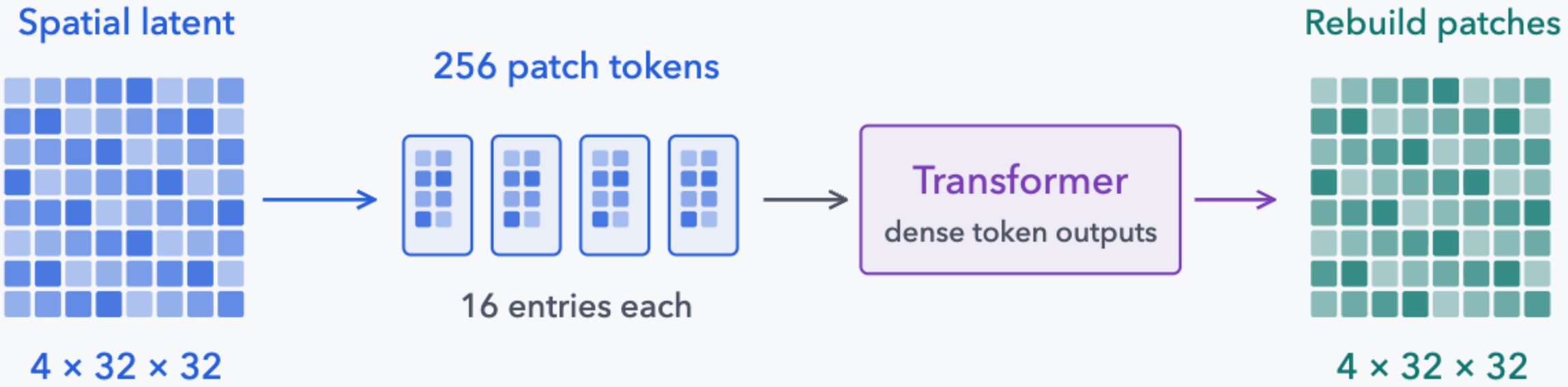
Takeaway The U-Net combines coarse features with finer encoder features and returns one noise value per pixel.

Inject the timestep throughout the denoiser



Takeaway Time features enter multiple blocks so the denoiser can adapt its processing to the noise level.

Can a transformer predict the noise?



Cells show patch structure schematically; no CLS pooling is used for this dense output.

Takeaway A diffusion transformer predicts patch entries that can be rearranged into a spatial output.

Why denoise a compressed representation?



3 × 512 × 512

Image encoder

compress spatially



4 × 64 × 64

64×

fewer positions

48×

fewer entries

Takeaway A smaller representation makes latent denoising attractive, but its size ratio is not a runtime measurement.

Train the autoencoder, then the denoiser

1 Learn the representation

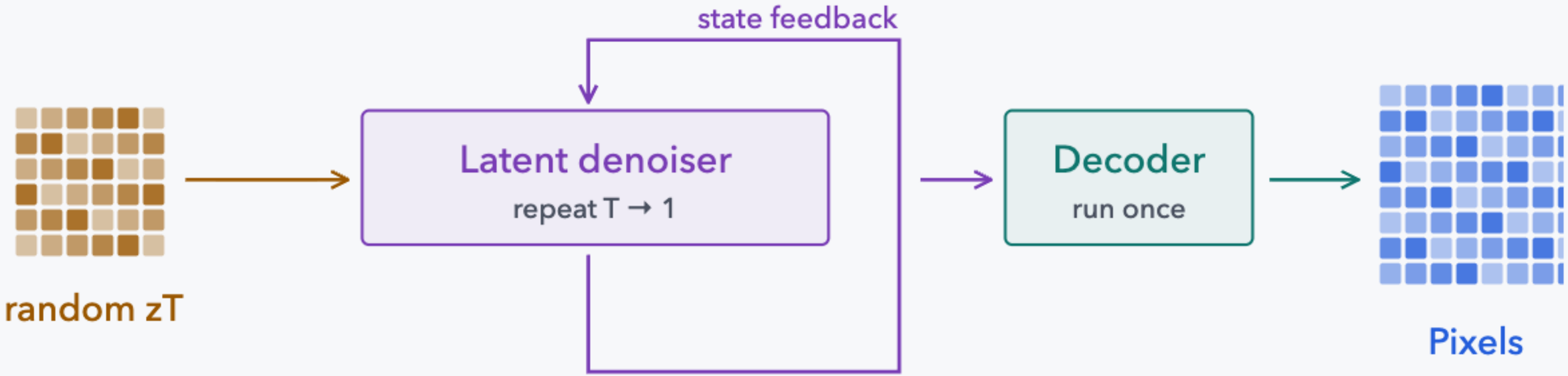


2 Learn to denoise latents



Takeaway Latent diffusion learns in a representation supplied by an autoencoder, with checkpoint-specific scaling.

Generate in latent space, decode once



For image-to-image, an encoder can prepare a supplied image first.

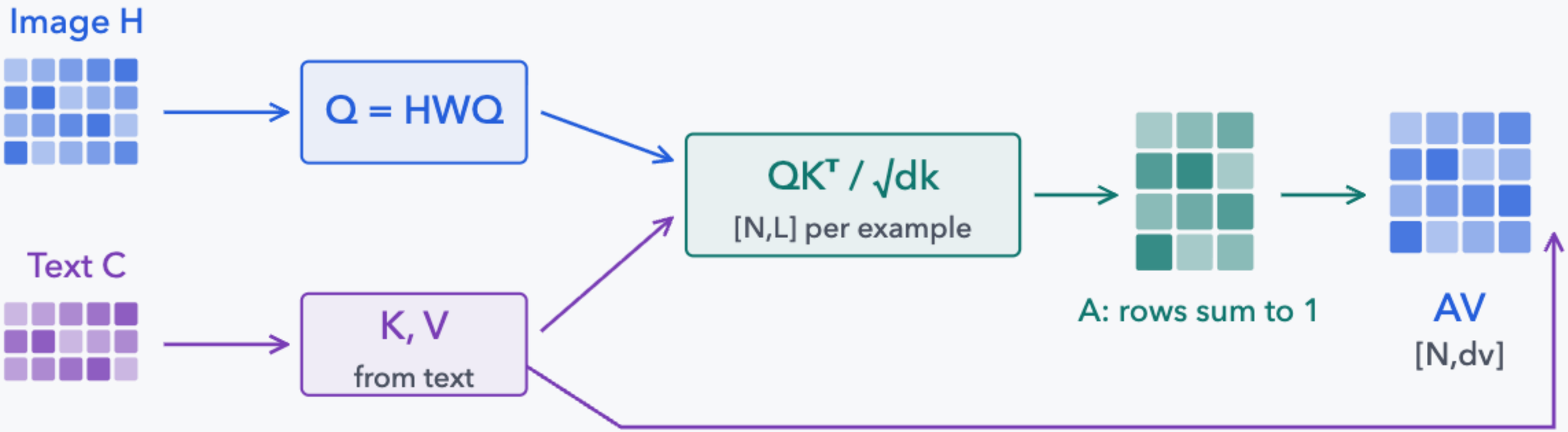
Takeaway The standard sampling path denoises in latent space, then decodes the final latent into pixels once.

What information comes from the prompt?



Takeaway Text conditioning supplies a feature sequence to the denoiser throughout generation.

How do image features attend to text?



Takeaway Each image query weights text values, so the result keeps one row per image position.

Work through one attention calculation

View: Attention arithmetic | Image query: Query 1: [1,0] | Stage | Guidance s | Reset | Step | Play | Pause

	golden	dog	grass
K	[1,0]	[0,1]	[1,1]
V	[1,0]	[0,2]	[1,1]



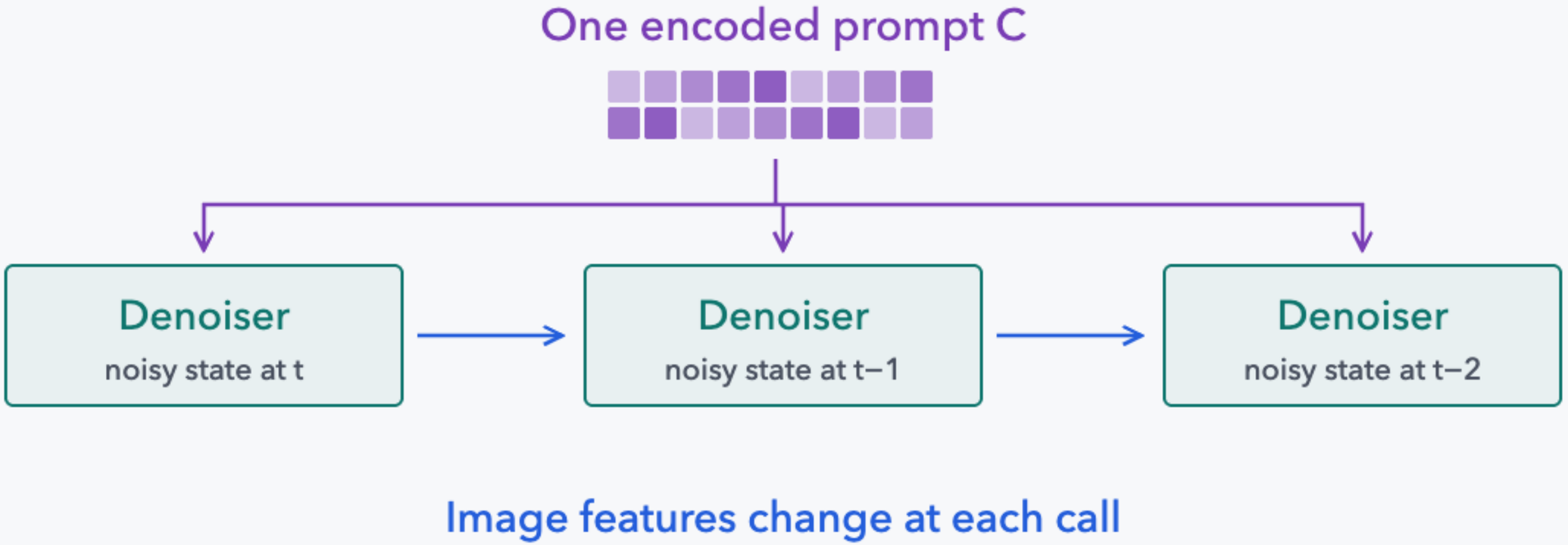
A	0.4011	0.1978	0.4011

```
QKᵀ = [1,0,1]
Scale: [0.7071,0.0000,0.7071]
Row sums to 1.000000
AV = [0.802224,0.796664]
```

Hand-designed numerical features, not recovered model attention.

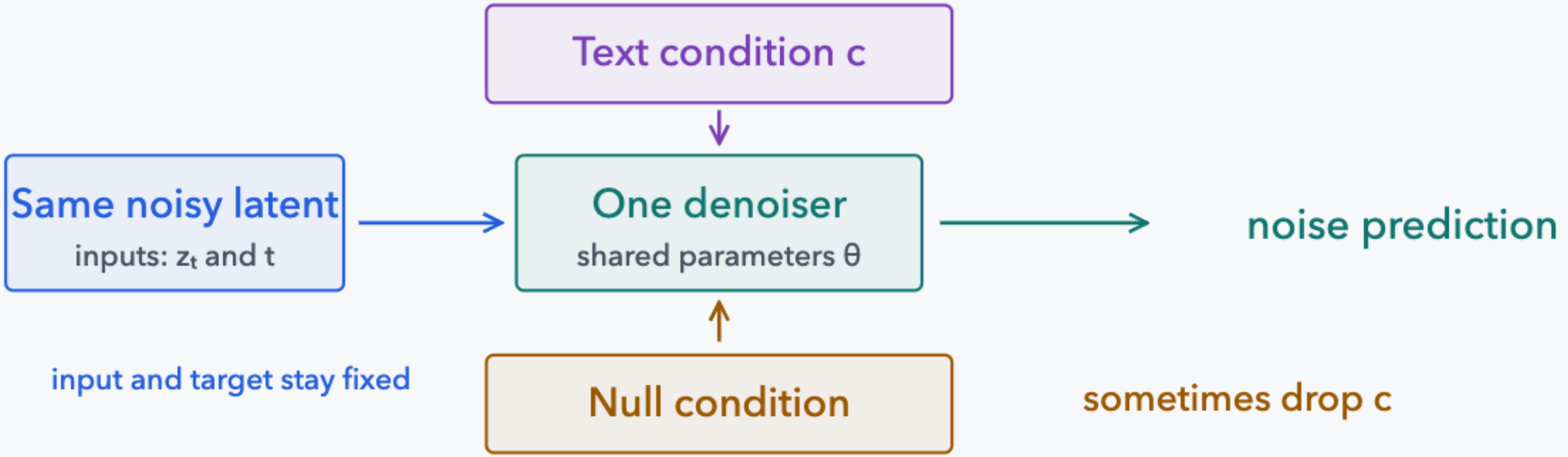
Takeaway Attention turns similarity scores into weights whose row sums to one, then uses them to combine values.

Use the prompt at every denoising step



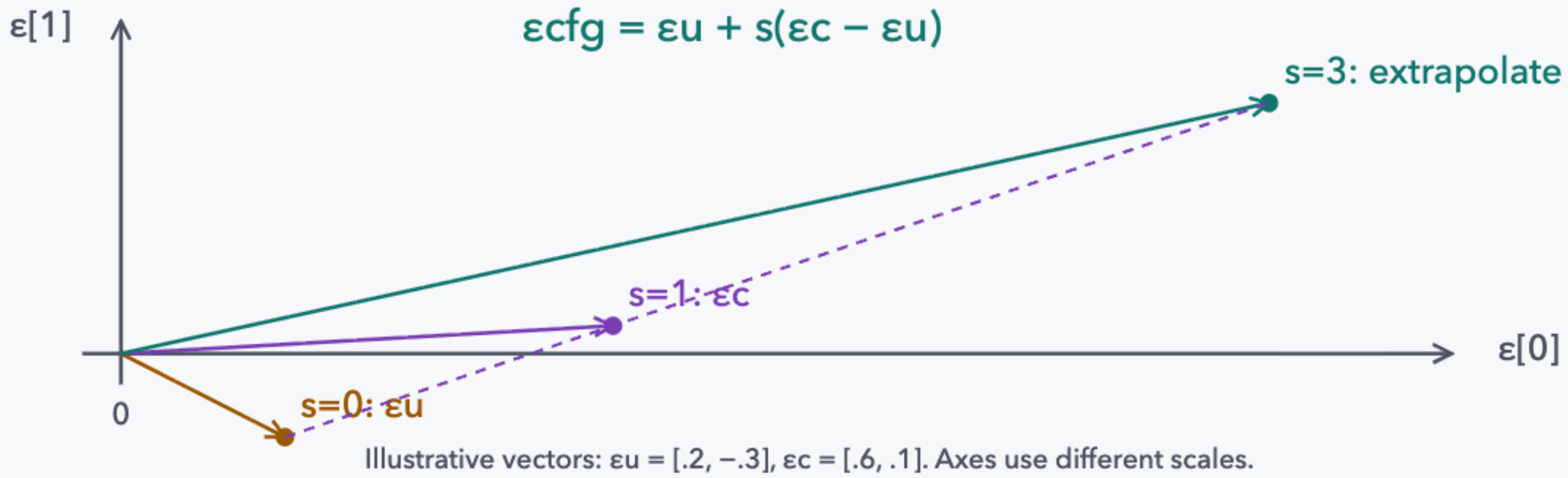
Takeaway Cross-attention brings text information into repeated denoising calls.

Teach one model to work with or without a condition



Takeaway Condition dropout trains the same model to make both conditional and unconditional predictions.

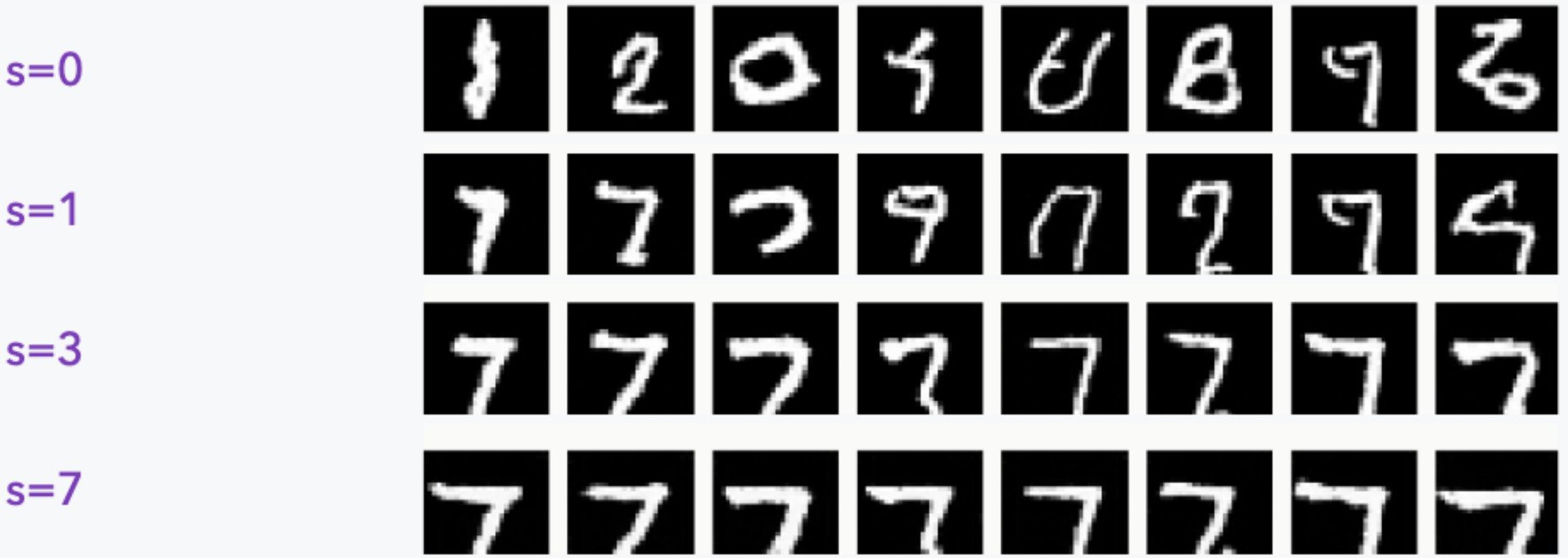
Combine predictions with classifier-free guidance



Illustrative vectors: εu = [.2, −.3], εc = [.6, .1]. Axes use different scales.

Takeaway A scale of 1 gives the conditional prediction; larger scales extrapolate beyond it.

What changes when we turn up guidance?



Request: digit 7 · identical random draws across rows · old checkpoint, seed 1002

Takeaway Guidance changes the samples, but a larger scale does not guarantee better quality.

Revisit completion and super-resolution

INPAINTING



Known region + missing region

SUPER-RESOLUTION



Low-resolution observation + inferred detail

Published RePaint and SR3 examples; each method has its own observation model.

Takeaway Completion and super-resolution need sampling procedures that respect their particular observations.

What do these results establish?

One unselected batch



2,000
training steps

667
sampling seed

Not computed
quality metrics

> **Takeaway** This run demonstrates the pipeline; its malformed and similar digits show that sample quality remains limited.

Why not add Gaussian noise to token IDs?

Sentence	The	dog	chased	the	ball
IDs A	8	24	3	9	11
IDs B	501	2	83	17	4
Masking	The	?	chased	?	ball

Takeaway Token IDs are arbitrary labels; masking is one way to define meaningful discrete corruption.

Build a masked-text training example

Mask probability t

Seed 667

Mode: Training corruption

Iteration

Reset | Step | Play | Pause

ORIGINAL SEQUENCE

The	dog	chased	the	ball

CORRUPTED INPUT

[MASK]	[MASK]	[MASK]	[MASK]	ball

TRAINING LOSS

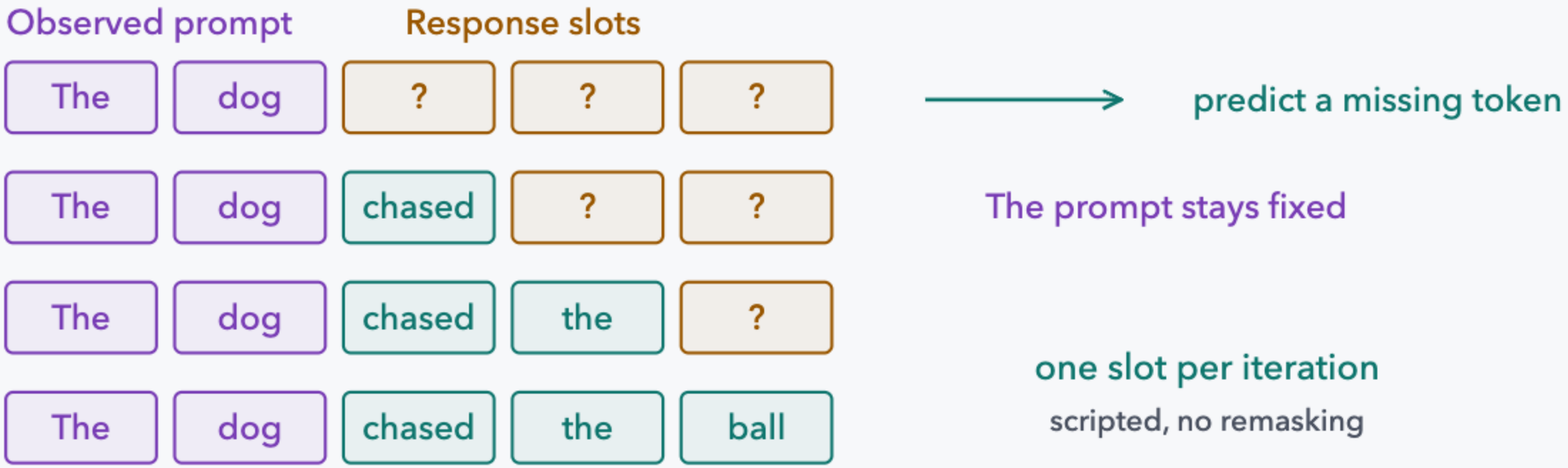
CE(The)	CE(dog)	CE(chased)	CE(the)	–

```
t = 0.5; seed 667
Masked positions: 4
Logits: [B,5,V]
Illustrative p(true token)=.7
Masked CE sum=1.4267
Mean CE=0.3567
LLaDA-style time weight is 1/t;
this demo shows unweighted CE.
```

Illustrative masked-text construction and scripted trace; no language model inference.

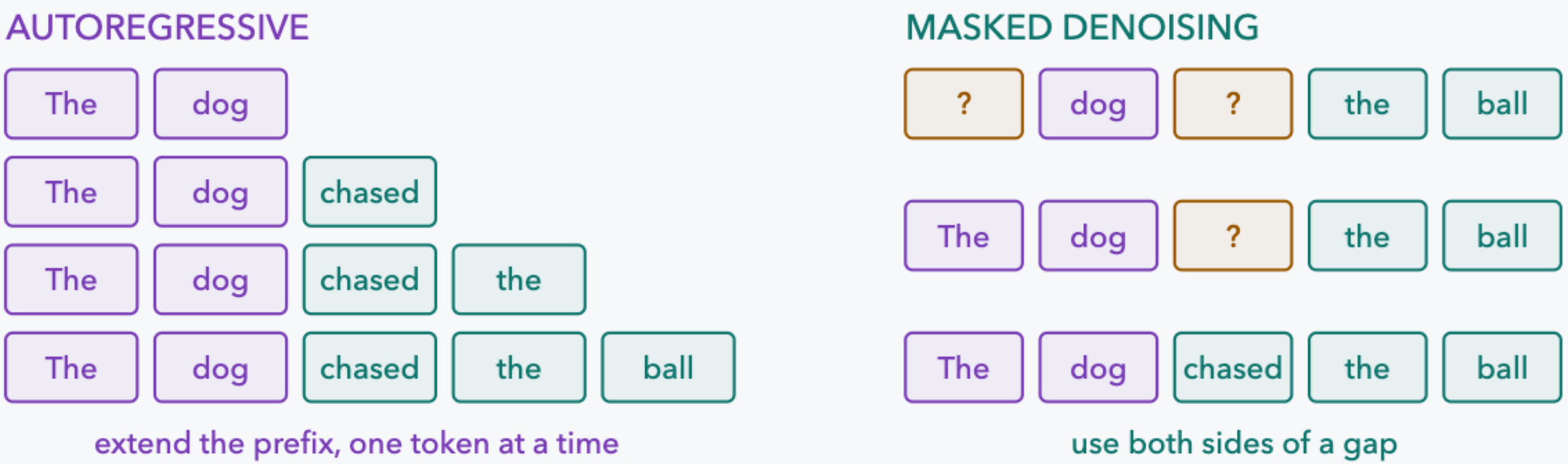
Takeaway Masked-text training learns missing-token predictions; the LLaDA-style objective also weights the loss by 1/t.

Fill the response a few tokens at a time



Takeaway An iterative text sampler needs a rule for which positions to fill, when to fill them and how long to generate.

How do the two text samplers differ?



Takeaway Parallel token predictions allow a different generation schedule; latency depends on the full implementation.

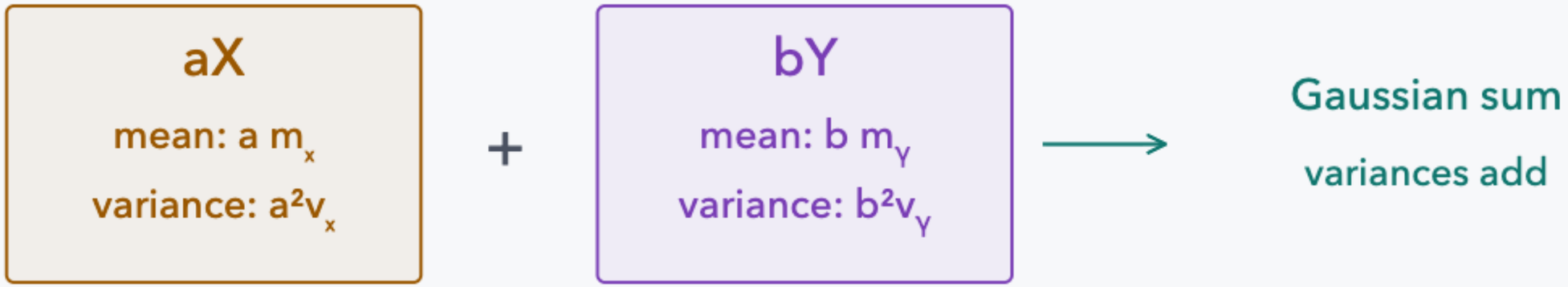
Can we now explain the opening examples?

State	Condition	Training target	Sampling rule
Pixels	partial image	known noise	respect observed region
Latents	prompt features	latent noise	denoise, then decode
Tokens	visible context	missing tokens	fill response slots

A comparison of the teaching constructions; full systems can use other targets and samplers.

Takeaway Training learns a prediction from corrupted examples; generation repeatedly uses that prediction to draw a sample.

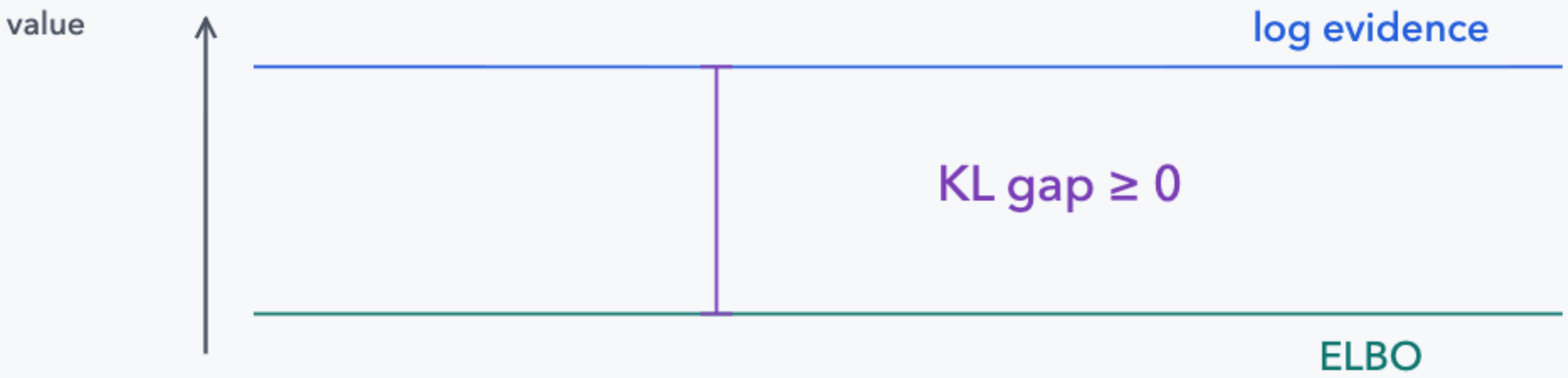
Independent Gaussians: means and variances



$$aX + bY \sim \mathcal{N}(am_x + bm_y, a^2v_x + b^2v_y)$$

Takeaway Independent Gaussian terms combine into a Gaussian whose variance is the sum of the scaled variances.

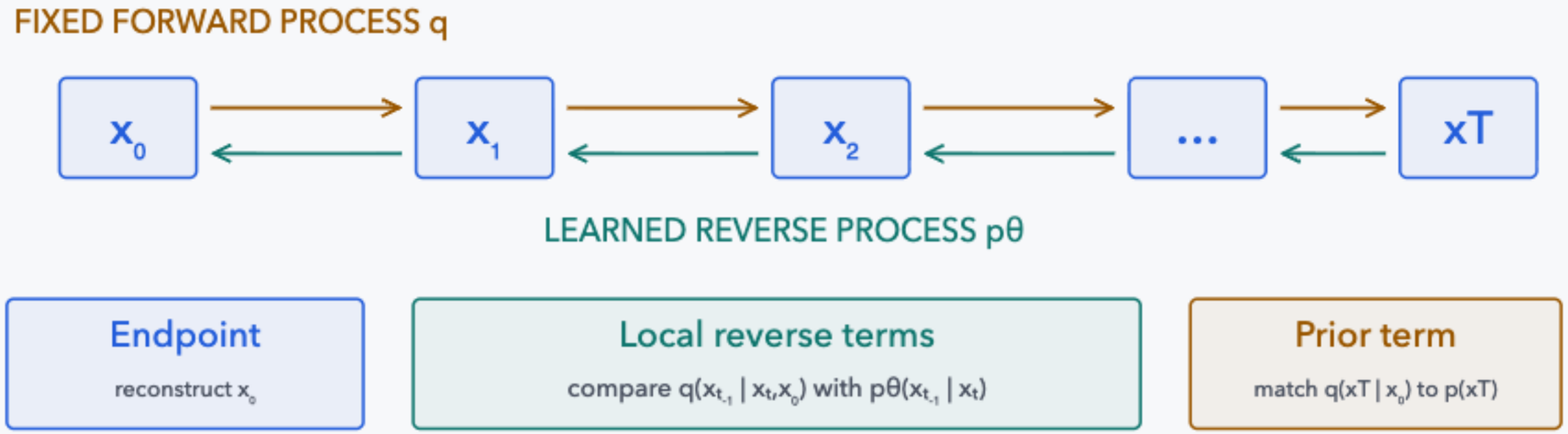
A lower bound on log evidence



$$\log p_\theta(x_0) = \mathrm{ELBO} + D_{KL}(q(x_{1:T}|x_0) \| p_\theta(x_{1:T}|x_0))$$

Takeaway The gap between log evidence and the ELBO is a nonnegative posterior KL divergence.

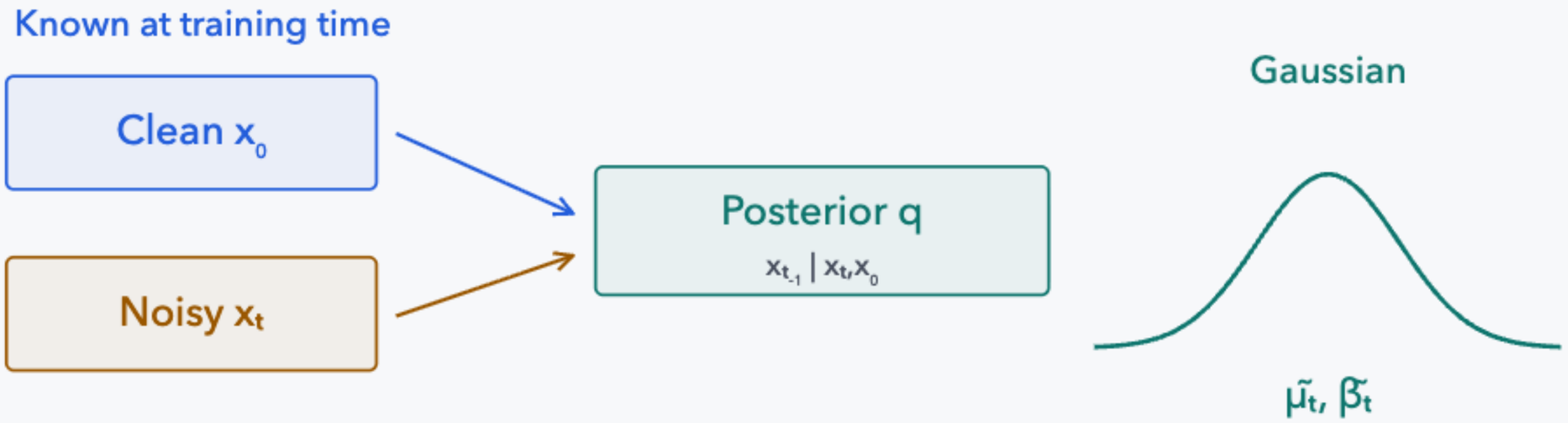
Factor the fixed chain backwards



$$\mathcal{L}_{VLB} = D_{KL}(q(x_T|x_0)\|p(x_T)) - \mathbb{E}_q \log p_\theta(x_0|x_1)$$
$$+ \sum_{t=2}^{T} \mathbb{E}_q D_{KL}(q(x_{t-1}|x_t, x_0)\|p_\theta(x_{t-1}|x_t))$$

Takeaway This factorisation separates the negative bound into prior matching, reverse matching and reconstruction.

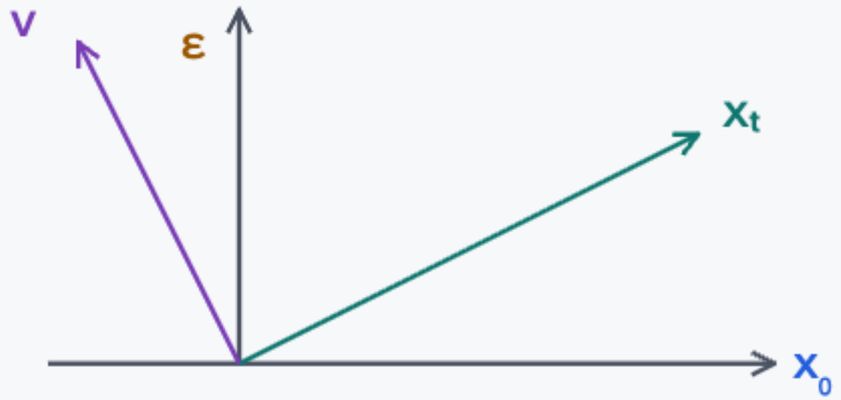
The tractable training posterior



$$\tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$$

> **Takeaway** Conditioning on both the noisy state and clean image gives a tractable training posterior.

Alternative prediction targets



Forward: rotate (x_0, ε)

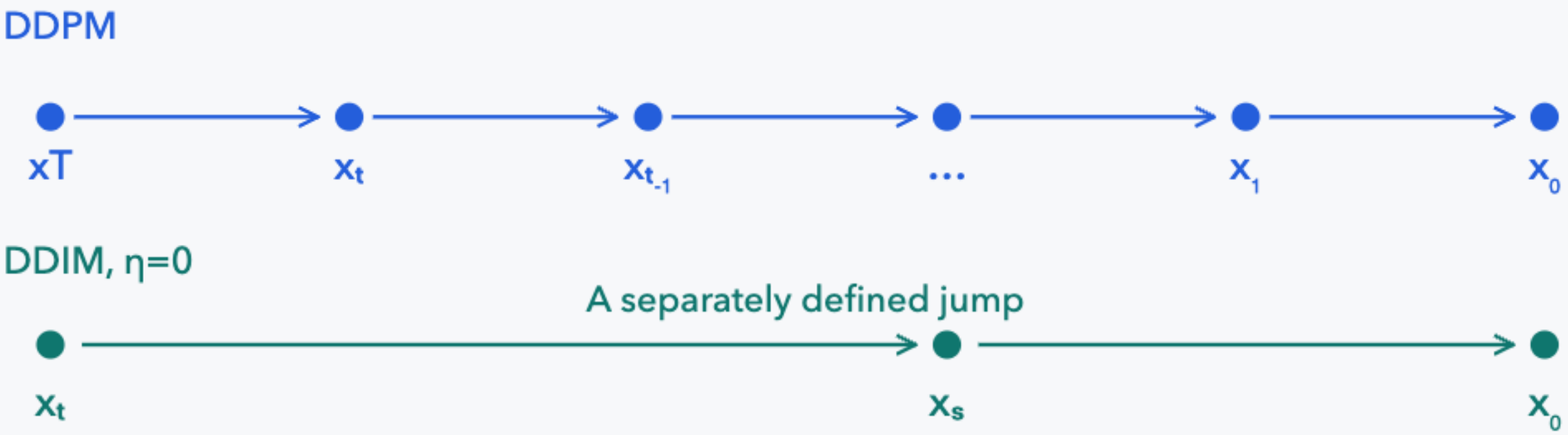
$\bar{\alpha}_t + (1-\bar{\alpha}_t) = 1$

Inverse: transpose the rotation

$$\epsilon = \sqrt{1-\bar{\alpha}_t}x_t + \sqrt{\bar{\alpha}_t}v$$

Takeaway Prediction targets can be converted, but the loss, sampler and checkpoint must use matching conventions.

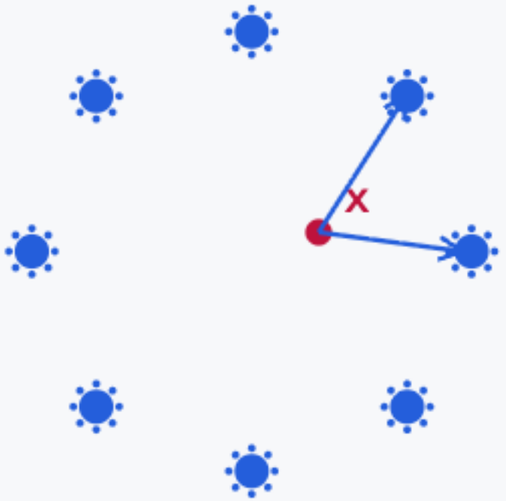
A faster sampler needs a different transition



$$x_s = \sqrt{\bar{\alpha}_s}\hat{x}_0 + \sqrt{1-\bar{\alpha}_s}\hat{\epsilon} \quad (s < t,\ \text{DDIM}\ \eta = 0)$$

Takeaway DDIM defines a different transition that can connect selected timesteps.

A score oracle for eight Gaussian clusters



Responsibilities r_k

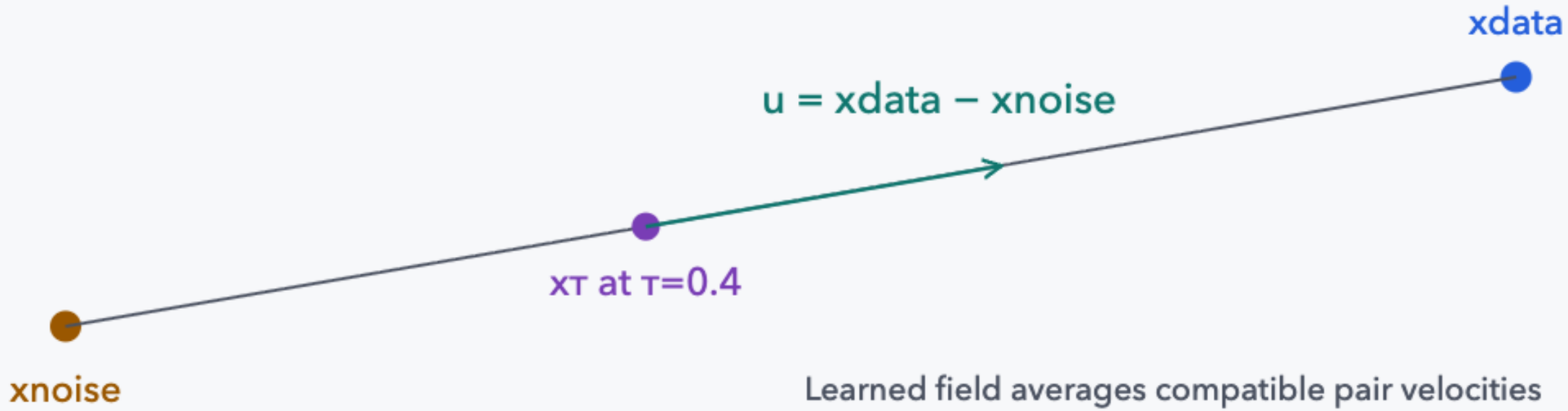
nearby centres get more weight → $\Sigma\, r_k\, \text{score}_k$

mixture score

$$\nabla_x \log q_t(x) = \sum_k r_k \frac{m_{k,t} - x}{v_t}, \quad \epsilon^* = -\sqrt{1-\bar{\alpha}_t}\nabla_x \log q_t(x)$$

Takeaway The mixture score is exact here, while the displayed finite-step Gaussian reverse remains approximate.

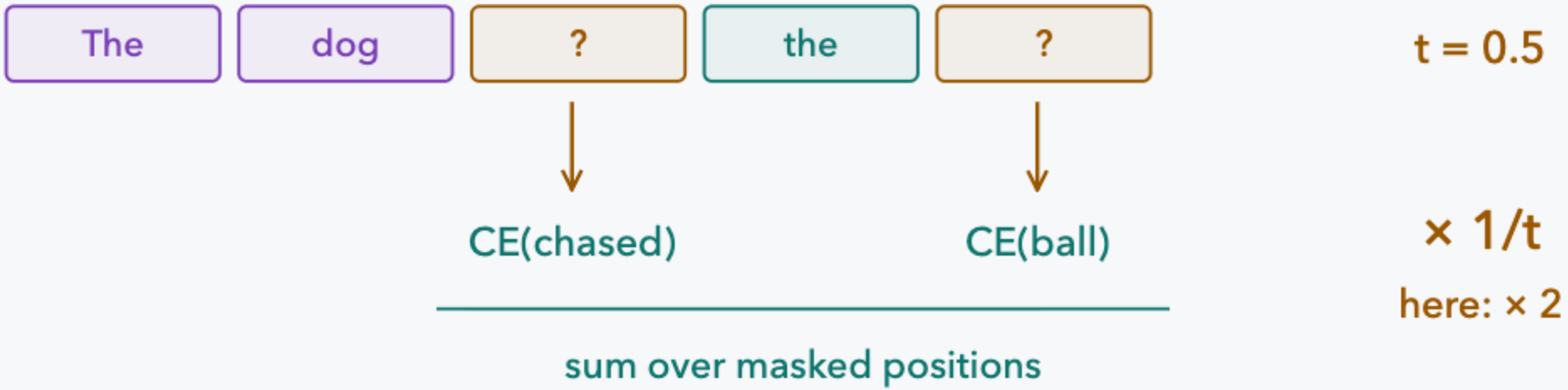
Flow matching: learn a velocity field



$$\frac{dx_\tau}{d\tau} = v_\theta(x_\tau, \tau), \qquad x_{\tau=0} \sim p_{noise}$$

Takeaway Flow matching learns a velocity field; sampling integrates that field from noise towards data.

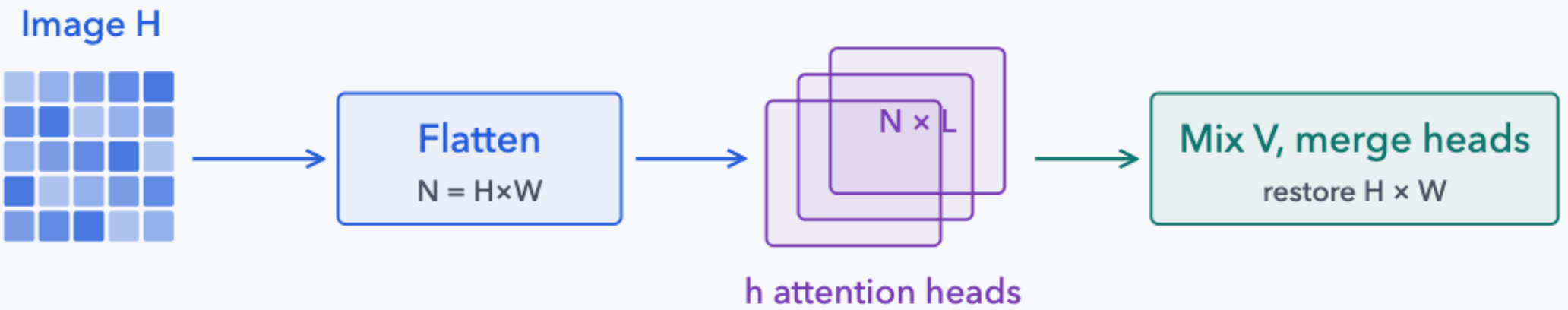
Masked language: objective and sampling convention



$$\mathcal{L} = -\mathbb{E}_{t,x_0,x_t}\left[\frac{1}{t}\sum_i \mathbf{1}[x_t^i = MASK]\log p_\theta(x_0^i|x_t)\right]$$

> **Takeaway** The time weighting is part of this objective, so unweighted masked cross-entropy is a different loss.

Cross-attention with heads and spatial layout



$$A : [B, h, N, L], \quad AV : [B, h, N, d_v] \rightarrow [B, N, hd_v]$$

Takeaway Cross-attention normalises over text positions for each image query in each head.

What to report with a generated example

Published	Exact source + supplied condition	Source figure
Local	Checkpoint + seed + sampler + budget	Recorded run
Proposed	Observation model + validation plan	No measured result

Takeaway A reproducible example needs its condition, model, sampling settings and evidence of what was evaluated.